ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Khoa Công nghệ Thông tin — Bộ môn Công nghệ Tri thức

MÔN HỌC: NHẬP MÔN MÃ HÓA - MẬT MÃ

BÁO CÁO ĐỒ ÁN 1

SỐ NGUYÊN TỐ & TRAO ĐỔI KHÓA DIFFIE-HELLMAN

Giảng viên phụ trách:	Thầy Trần Hải Anh Điền
Nhóm thực hiện:	**Nhóm 18**

DANH SÁCH THÀNH VIÊN NHÓM:

STT	Mã số sinh viên	Họ và tên	Tỷ lệ đóng góp
1	**24120070**	**Vương Hữu Khang**	**33.3%**
2	**24120089**	**Tạ Quang Mạnh**	**33.3%**
3	**24120091**	**Lâm Duy Minh**	**33.3%**

THÀNH PHỐ HỒ CHÍ MINH — THÁNG 11 / 2026

MỤC LỤC TỔNG QUAN

Mục 1: Hệ thống xử lý số lớn (Big Integer) **Trang 3**

1.1. Cấu trúc dữ liệu tĩnh 2048-bit (big_t)

1.2. Phép toán số học cơ bản (+, -, *) và xử lý nhớ 128-bit

1.3. Phép chia nhiều từ Knuth Algorithm D (divmod, /, %)

1.4. Lũy thừa modulo & Các phép toán hỗ trợ (AddMod, MultMod, ExpMod)

Mục 2: Thuật toán kiểm tra số nguyên tố & Sinh Safe Prime **Trang 8**

2.1. Phân tích & So sánh các thuật toán (Chia thử, Fermat, Solovay-Strassen, AKS, Miller-Rabin)

2.2. Số nguyên tố an toàn (Safe Prime) & Phòng chống tấn công Pohlig-Hellman, nhóm con nhỏ

2.3. Kiến trúc lọc 4 tầng & Sàng đồng thời 302 số nguyên tố nhỏ

2.4. Kết quả thực nghiệm Benchmark sinh Safe Prime

Mục 3: Số ngẫu nhiên trong Mật mã học (Mục điểm cộng) **Trang 13**

3.1. Vai trò của số ngẫu nhiên & Phân loại TRNG, PRNG, CSPRNG

3.2. Vì sao cấm sử dụng rand() và mt19937 trong mật mã

3.3. Kiến trúc CSPRNG: Nguồn Entropy OS (rand_s / getrandom) + Keccak SHAKE128

3.4. Rejection Sampling triệt tiêu Modulo Bias & Các sự cố bảo mật lịch sử

Mục 4: Ứng dụng thực tế của trao đổi khóa Diffie-Hellman **Trang 17**

4.1. Diffie-Hellman trong TLS 1.3 / HTTPS & Tính bí mật chuyển tiếp (PFS)

4.2. Các cuộc tấn công thực tế (Man-in-the-Middle, Tấn công Logjam CVE-2015-4000)

4.3. Xu hướng tiến hóa: ECDHE trên đường cong Elliptic & Mật mã hậu lượng tử

Mục 5: Mô tả chương trình, kiểm thử & Hướng dẫn sử dụng **Trang 21**

5.1. Môi trường phát triển, chuẩn C++14 & Kiến trúc Unity Build (*.cpp)

5.2. Hướng dẫn biên dịch và tùy biến tham số dòng lệnh (--bits, --rounds, --seed)

5.3. Minh họa kết quả chạy thực tế của chương trình

Mục 6: Tự đánh giá ưu/nhược điểm, hướng cải thiện & Phân công **Trang 25**

6.1. Đánh giá phần PRNG, Miller-Rabin & Sinh Safe Prime

6.2. Đánh giá phần Big Integer & Thuật toán chia Knuth D

6.3. Đánh giá phần Giao thức Diffie-Hellman & Lũy thừa Modulo

6.4. Bảng tổng hợp mức độ hoàn thành & Đóng góp của từng thành viên

MỤC 1: HỆ THỐNG XỬ LÝ SỐ LỚN (BIG INTEGER)

Phần bổ sung (Chia/Mod & Lũy thừa mô-đun): Tạ Quang Mạnh (MSSV: 24120089)

1.1. Đặt vấn đề và Mục tiêu thiết kế

Trong giao thức trao đổi khóa Diffie-Hellman, các phép toán mật mã được thực hiện trên trường hữu hạn $\mathbb{Z}_p^*$ với số nguyên tố an toàn p có kích thước tối thiểu là 512 bit (hoặc 1024, 2048 bit trong thực tế). Các kiểu dữ liệu nguyên thủy trong chuẩn C++14 (như `uint64_t`) chỉ hỗ trợ tối đa 64 bit, hoàn toàn không đáp ứng được yêu cầu tính toán.

Hơn nữa, quy định của đề bài nghiêm cấm việc sử dụng các thư viện số lớn bên ngoài (như GMP, NTL, Boost.Multiprecision). Do đó, nhóm bắt buộc phải tự thiết kế và cài đặt cấu trúc số nguyên lớn từ đầu.

Mục tiêu thiết kế của kiểu số lớn `uint_t<N>`:

1. **Không âm (Unsigned)**: Toàn bộ các giá trị trong Diffie-Hellman (số nguyên tố p, phần tử sinh g, khóa riêng a, b, khóa công khai A, B, và bí mật chung s) đều là các số nguyên dương nằm trong khoảng $[0, p-1]$. Việc chỉ xử lý số không âm giúp tối ưu hóa đáng kể độ phức tạp của các thuật toán và tránh chi phí xử lý bit dấu.
2. **Độ rộng cố định (Fixed-width) qua Template**: - Sử dụng `template <size_t N>` với N là số lượng limb 64-bit. - Bộ nhớ của số lớn được cấp phát tĩnh (stack/inline), không dùng cấp phát động (`std::vector` hay `malloc/new`). Điều này giúp triệt tiêu chi phí phân mảnh bộ nhớ (heap fragmentation), tận dụng tối đa bộ nhớ đệm CPU (L1/L2 cache locality) và tăng tốc độ thực thi đáng kể trong các vòng lặp tính lũy thừa lặp lại hàng nghìn lần.
3. **Quy tắc gấp đôi độ rộng (`MAX_PRIME_BITS`)**: - Trong phép nhân hai số $a, b < p$ (với p có tối đa B bit), tích $a \times b$ có độ dài tối đa là $2B$ bit trước khi thực hiện phép chia lấy dư $\bmod p$. - Do đó, nhóm định nghĩa kiểu làm việc chung `big_t` có độ rộng gấp đôi kích thước bit tối đa của p:

$$\text{BIG_LIMBS} = \frac{2 \times \text{MAX_PRIME_BITS}}{64}$$

Với $\text{MAX_PRIME_BITS} = 1024$ bit, nhóm chọn $N = 32$ limbs (tương đương 2048 bit). Nhờ đó, mọi phép cộng, trừ, nhân đều không bao giờ bị tràn số (overflow) ngoài tầm kiểm soát.

1.2. Cấu trúc biểu diễn dữ liệu

Cấu trúc được định nghĩa trong file `project_01_source/bigint.cpp`:

```
__extension__ typedef unsigned __int128 uint128_t;

template <size_t N>
struct uint_t {
    uint64_t digits[N]; // Mảng N phần tử 64-bit
    ...
};
```

- **Thứ tự lưu trữ (Endianness)**: Little-endian theo cấp độ limb:
- `digits[0]` lưu 64 bit có trọng số thấp nhất ($2^0 \to 2^{63}$).
- `digits[1]` lưu 64 bit tiếp theo ($2^{64} \to 2^{127}$).
- `digits[N - 1]` lưu 64 bit cao nhất.
- Giá trị toán học của số nguyên được biểu diễn là:

$$A = \sum_{i=0}^{N-1} \text{digits}[i] \times 2^{64i}$$

- **Kiểu dữ liệu mở rộng** `uint128_t`: Trình biên dịch `g++` (GCC/Clang) hỗ trợ kiểu mở rộng phần cứng 128-bit `__uint128_t`. Kiểu này cho phép nhân hai số 64-bit hoặc cộng hai số 64-bit có nhớ để cho ra kết quả 128-bit chỉ trong một vài chu kỳ xung nhịp CPU mà không cần phải bẻ nhỏ thành các limb 32-bit, giúp hiệu năng tăng gấp 3 đến 4 lần.

1.3. Chi tiết các thuật toán và độ phức tạp

1.3.1. Các hàm tiện ích & Thao tác bit

- `is_zero()`: Duyệt kiểm tra mọi limb xem có bằng 0 hay không. Độ phức tạp: $O(N)$.
- `is_odd()`: Kiểm tra bit thấp nhất: `(digits[0] & 1) != 0`. Độ phức tạp: $O(1)$.
- `test_bit(size_t i)` & `set_bit(size_t i)`: Xác định limb chứa bit: $\text{limb} = i/64$, và vị trí bit trong limb: $\text{bit} = i \bmod 64$. Độ phức tạp: $O(1)$.
- `bit_length()`: Tìm limb cao nhất khác 0 (`top`). Sau đó sử dụng hàm nội tại của GCC `__builtin_clzll` (Count Leading Zeros) để xác định số bit có nghĩa trong limb cao nhất:

$$\text{bit_length} = \text{top} \times 64 + (64 - \text{clzll}(\text{digits}[\text{top}]))$$

  Độ phức tạp: $O(N)$.

1.3.2. Chuyển đổi chuỗi (I/O)

- `from_hex(const std::string &s)`:
- Tự động bỏ qua tiền tố `0x` hoặc `0X` (nếu có).
- Duyệt chuỗi hex từ phải qua trái, gom từng cụm 16 ký tự hex vào một limb 64-bit (`digits[0]`, `digits[1]`,...).
- Độ phức tạp: $O(L)$ với L là độ dài chuỗi ký tự.
- `to_hex()`:
- Tìm limb cao nhất khác 0, in limb này không đệm số 0 đầu (`%llx`).
- Các limb thấp hơn được in với đúng 16 ký tự có đệm số 0 (`%016llx`).
- Nếu số bằng 0, trả về `"0"`.
- Độ phức tạp: $O(N)$.
- `to_dec()`:
- Để chuyển đổi số nhị phân lớn sang hệ thập phân một cách nhanh chóng, nhóm không chia từng số 10 đơn lẻ mà thực hiện phép chia lặp cho 10^{18} (lũy thừa lớn nhất của 10 nằm vừa trong `uint64_t`):
  - Mỗi vòng lặp, số lớn được chia cho 10^{18} từ limb cao xuống thấp bằng `uint128_t`.
  - Phần dư 64-bit thu được chính là 18 chữ số thập phân tương ứng.
- Kỹ thuật này giảm số bước chia tới 18 lần.
- Độ phức tạp: $O(N^2)$.

1.3.3. Các phép toán so sánh

- `operator==`: So sánh từng limb từ 0 đến $N-1$. Độ phức tạp: $O(N)$.
- `operator<`: So sánh từ limb cao nhất $N-1$ xuống limb thấp nhất 0. Ngay khi gặp cặp limb khác nhau, đưa ra kết luận. Độ phức tạp: $O(N)$ trong trường hợp xấu nhất, trung bình $O(1)$.
- Các toán tử khác (`!=`, `>`, `<=`, `>=`) được suy ra trực tiếp từ `==` và `<`.

1.3.4. Phép cộng (`operator+`)

- Thuật toán cộng có nhớ (Ripple-carry Addition): `cpp uint128_t carry = 0; for (size_t i = 0; i < N; i++) { uint128_t sum = (uint128_t)a.digits[i] + b.digits[i] + carry; res.digits[i] = (uint64_t)sum; carry = sum >> 64; }`
- Kết quả được tự động lấy theo modulo 2^{64N}.
- Độ phức tạp: $O(N)$.

1.3.5. Phép trừ (`operator-`)

- Giả định tiền điều kiện: $a \geq b$.
- Thuật toán trừ có mượn:
- Nếu `a.digits[i] >= b.digits[i] + borrow`: không cần mượn, `borrow = 0`.
- Ngược lại: mượn 2^{64} từ limb kế tiếp, `borrow = 1`.
- Độ phức tạp: $O(N)$.

1.3.6. Phép nhân (`operator*`)

- Thuật toán nhân cổ điển (Schoolbook Multiplication):
- Nhóm tối ưu hóa bằng cách xác định số lượng limb có nghĩa thực tế của a (gọi là len_a) và b (gọi là len_b).
- Bỏ qua các limb bằng 0 trong vòng lặp ngoài:

$$\text{res}[i+j] \leftarrow \text{res}[i+j] + a[i] \times b[j] + \text{carry}$$

- Độ phức tạp thời gian: $O(len_a \times len_b) \leq O(N^2)$. Trong các phép tính với số nhỏ, thời gian giảm tỷ lệ thuận với số limb thực tế.

1.3.7. Phép dịch bit (`<<` và `>>`)

- Được phân tách thành hai giai đoạn: 1. Dịch chuyển limb nguyên: $\text{limb_shift} = \text{shift}/64$. 2. Dịch chuyển bit lẻ trong limb: $\text{bit_shift} = \text{shift} \bmod 64$.
- Sử dụng phép gộp bit giữa limb hiện tại và limb liền kề để dịch chính xác trong 1 lượt duyệt.
- Độ phức tạp: $O(N)$.

1.3.8. Sàng số nhỏ (`mod_small`)

- Tính $a \bmod m$ với m là số nguyên nhỏ 32-bit (dùng để sàng các số nguyên tố nhỏ trước khi chạy Miller-Rabin).
- Áp dụng sơ đồ Horner từ limb cao nhất xuống:

$$\text{rem} = ((\text{rem} \times 2^{64}) + \text{digits}[i]) \bmod m$$

- Độ phức tạp: $O(len_a)$.

1.3.9. So sánh hai hướng tiếp cận số học mô-đun (AddMod, MultMod vs. Bộ đệm $2n$-bit)

Dựa trên tài liệu tham khảo bổ sung của môn học (`pseudocode-modular-operation.pdf`), nhóm đã nghiên cứu và phân tích hai hướng tiếp cận khác nhau trong việc thực thi các phép toán mô-đun:

1. **Phương pháp 1: Giữ biến trung gian luôn trong n-bit (Mã giả của giảng viên)**: - **Cơ chế**:
   - `AddMod(x, y, p)`: Dùng điều kiện kiểm tra $x \geq p - y$ để tính $(x + y) \bmod p$ bằng phép trừ $x - (p - y)$, triệt tiêu nguy cơ tràn bit n.
   - `MultMod(x, y, p)`: Tính $(x \cdot y) \bmod p$ bằng kỹ thuật cộng-dồn nhị phân (Double-and-Add) kết hợp `AddMod`, duyệt qua n bit của y.
   - `ExpMod(x, y, p)`: Lũy thừa mô-đun dạng Square-and-Multiply dựa trên `MultMod`.
   - **Ưu điểm**: Mọi biến trung gian được đảm bảo không vượt quá n-bit. Rất phù hợp nếu hệ thống bị ép buộc chỉ được cấp phát một struct kích thước cố định đúng n-bit (như `uint512_t` thuần túy).

- **Hạn chế**: Mỗi phép nhân `MultMod` phải lặp qua toàn bộ số bit của y (512 lượt gọi `AddMod`), dẫn đến độ phức tạp thực thi cao hơn.

2. **Phương pháp 2: Sử dụng bộ đệm độ rộng gấp đôi ($2n$-bit) kết hợp nhân khối (Cách nhóm cài đặt)**: - **Cơ chế**:
   - Định nghĩa `big_t` có độ rộng $2n$-bit (2048 bit).
   - Phép nhân hai số n-bit $x \times y$ được thực hiện nguyên khối qua thuật toán Schoolbook tận dụng lệnh nhân 128-bit phần cứng `uint128_t` (chỉ mất $8 \times 8 = 64$ phép nhân limb cho 512-bit).
   - Sau đó áp dụng phép chia nhị phân `divmod` / `%` để lấy phần dư modulo p.
   - **Ưu điểm**: Tối ưu hóa sâu phần cứng CPU, tốc độ nhanh hơn vượt trội. Thực nghiệm đo đạc lũy thừa mô-đun 512-bit cho thấy:
   - Phương pháp 2 (Nhóm): **~20.1 ms**.
   - Phương pháp 1 (Mã giả thầy): **~62.8 ms** (chậm hơn khoảng 3 lần).
   - **Kết luận**: Nhóm cài đặt đầy đủ cả hai phương án trong mã nguồn (`dh.cpp`) để đối chiếu, kiểm chứng tính chính xác (cho ra kết quả khớp 100%) và phục vụ tối ưu hóa tốc độ.

1.4. Bảng tổng kết độ phức tạp các toán tử

Toán tử / Hàm	Mục đích	Độ phức tạp thời gian	Bộ nhớ phụ
`is_zero()`, `is_odd()`	Kiểm tra 0 / số lẻ	$O(N)$ / $O(1)$	$O(1)$
`bit_length()`	Đếm số bit có nghĩa	$O(N)$	$O(1)$
`test_bit()`, `set_bit()`	Thao tác bit đơn lẻ	$O(1)$	$O(1)$
`==`, `!=`, `<`, `>`, `<=`, `>=`	So sánh hai số lớn	$O(N)$	$O(1)$
`+`, `-`	Cộng, trừ số lớn	$O(N)$	$O(N)$
`*`	Nhân số lớn (Schoolbook)	$O(len_a \times len_b) \le O(N^2)$	$O(N)$
`<<`, `>>`	Dịch trái, dịch phải bit	$O(N)$	$O(N)$
`mod_small(a, m)`	Lấy dư với số nhỏ (sàng số)	$O(N)$	$O(1)$
`from_hex`, `to_hex`	Chuyển đổi chuỗi Hex	$O(N)$	$O(N)$
`to_dec`	Chuyển đổi chuỗi Dec	$O(N^2)$	$O(N)$
`divmod(a, b, q, r)`	Chia & lấy dư hai số lớn	$O(k \cdot N)$ (với k là số bit)	$O(N)$

1.5. Phép chia và lấy dư số lớn (`divmod`, `/`, `%`)

1.5.1. Vì sao phép chia là điểm nghẽn

Mọi phép nhân trong lũy thừa mô-đun đều phải rút gọn `mod p`. Với p 512 bit, một lần sinh số nguyên tố an toàn cần hàng nghìn lần Miller-Rabin, mỗi lần là một lũy thừa mô-đun (~768 phép nhân-mô-đun), nên tốc độ của `%` quyết định tốc độ toàn chương trình.

Phiên bản đầu dùng **chia nhị phân (shift-subtract)**: duyệt từng bit của số bị chia, mỗi bước dịch và so sánh/trừ cả số lớn, độ phức tạp $O(k \cdot N)$ với k là số bit của số bị chia. Đo thực tế: **72 µs** cho một phép `mulmod` 512 bit.

1.5.2. Thuật toán Knuth D (TAOCP tập 2, mục 4.3.1)

Thay vì xử lý từng bit, Knuth D tính **mỗi lần một limb 64-bit của thương**, giống phép chia dài ở tiểu học nhưng trong hệ cơ số $b = 2^{64}$.

Gọi số bị chia u có m limb, số chia v có n limb (bỏ các limb cao bằng 0):

1. **Chuẩn hóa (D1).** Dịch trái cả u và v đi $s = \mathrm{clz}(v_{n-1})$ bit để bit cao nhất của v_{n-1} bằng 1. Điều kiện $v_{n-1} \geq 2^{63}$ bảo đảm ước lượng ở bước sau lệch tối đa 2 đơn vị.
2. **Với mỗi $j = m-n, \ldots, 0$ (D2–D7):** - **Ước lượng (D3):** $\hat{q} = \lfloor (u_{j+n} \cdot 2^{64} + u_{j+n-1}) / v_{n-1} \rfloor$ dùng `uint128_t`. - **Hiệu chỉnh:** trong khi $\hat{q} \geq 2^{64}$ hoặc $\hat{q} \cdot v_{n-2} > \hat{r} \cdot 2^{64} + u_{j+n-2}$ thì giảm $\hat{q}$ đi 1 (lặp tối đa 2 lần). Bước này loại gần như mọi ước lượng thừa. - **Nhân-trừ (D4):** $u_{j..j+n} \leftarrow u_{j..j+n} - \hat{q} \cdot v$, dùng cờ mượn. - **Cộng bù (D5–D6):** nếu kết quả âm (xác suất khoảng $2/2^{64}$ với dữ liệu ngẫu nhiên) thì $\hat{q} \leftarrow \hat{q} - 1$ và cộng lại v. - Gán $q_j = \hat{q}$.
3. **Khử chuẩn hóa (D8):** phần dư $r = (u_{0..n-1}) \gg s$.

Trường hợp số chia chỉ có 1 limb ($n = 1$) dùng đường ngắn: chia tuần tự từ limb cao xuống thấp bằng `uint128_t`.

1.5.3. Độ phức tạp và kết quả đo

Số vòng ngoài là $m - n + 1$, mỗi vòng $O(n)$ nên tổng là $O(n(m-n+1))$ phép nhân 64×64→128. Trong `modular_exponentiation`, số bị chia là tích $2n$ limb và số chia n limb nên mỗi phép `%` tốn $O(n^2)$, tương đương một phép nhân.

Phép đo (g++ -O2)	Chia nhị phân	Knuth D
`mulmod` 512 bit	72 µs	0,44 µs
`modular_exponentiation`, mũ 512 bit	~55 ms	0,24 ms
`modular_exponentiation`, mũ 768 bit	–	0,63 ms
`modular_exponentiation`, mũ 1024 bit	–	1,17 ms

Nhanh hơn khoảng **160 lần**, vượt xa mục tiêu ≥ 10 lần của nhóm.

Cập nhật bảng 1.4: `divmod(a, b, q, r)` — Chia & lấy dư — $O(n(m-n+1))$ — bộ nhớ phụ $O(N)$.

1.5.4. Kiểm thử tính đúng đắn

- `tests/crosscheck_bigint.py`: 30.000 phép tính ngẫu nhiên đối chiếu với số nguyên Python, **0 sai lệch**.
- `tests/crosscheck_div_stress.py`: 50.016 phép chia/mod với các số "khó" (limb toàn 0, 2^{63}, $2^{64}-1$, …). Đo bằng bộ đếm tạm, bộ test này đi qua nhánh **cộng bù 3.242 lần** và nhánh hiệu chỉnh $\hat{q}$ 53.062 lần, **0 sai lệch**. (Test ngẫu nhiên thường gần như không bao giờ chạm nhánh cộng bù nên cần bộ test riêng này.)

1.6. Lũy thừa mô-đun (`modular_exponentiation`)

1.6.1. Bài toán

Tính $r = b^e \bmod m$ với e cỡ 512 bit. Nhân e lần là bất khả thi ($e \approx 2^{512}$).

1.6.2. Square-and-Multiply

Viết $e = \sum_{i=0}^{k-1} e_i 2^i$ ($e_i \in \{0,1\}$, k = số bit). Duyệt bit từ cao xuống thấp:

```
result = 1
for i = k-1 down to 0:
    result = result * result mod m          # bình phương
    if e_i == 1: result = result * b mod m  # nhân
```

Ví dụ $3^{13} \bmod 7$: $13 = 1101_2$. Khởi tạo 1; bit 1: $1^2 \cdot 3 = 3$; bit 1: $3^2 \cdot 3 = 27 \equiv 6$; bit 0: $6^2 = 36 \equiv 1$; bit 1: $1^2 \cdot 3 = 3$. Kết quả 3 (đúng: $3^{13} = 1594323 = 7 \cdot 227760 + 3$).

1.6.3. Vì sao hiệu quả với số lớn

- **Số phép nhân:** k phép bình phương và trung bình $k/2$ phép nhân, tức $\approx 1{,}5k$ phép `mulmod` (~768 với $k = 512$) thay vì 2^{512}.
- **Kích thước bị chặn:** rút gọn `mod m` ngay sau mỗi phép nhân nên mọi giá trị trung gian $< m$ và tích $< m^2$, vừa trong `big_t` (rộng gấp đôi `MAX_PRIME_BITS`, xem mục 1.1). Không bao giờ phải lưu số b^e khổng lồ.
- **Tổng độ phức tạp:** $O(k \cdot M(n))$ với $M(n) = O(n^2)$ là chi phí một phép nhân + chia n limb. Với 512 bit: $O(512 \cdot 8^2)$ phép nhân limb.

1.6.4. Trường hợp biên đã xử lý

`mod = 1 → 0`; `mod = 0 → 0` (không xác định, tránh chia 0); `exp = 0 → 1`; `base ≥ mod` được rút gọn trước; `base = 0`.

1.6.5. Kiểm thử

`tests/crosscheck_modexp.py` đối chiếu 4.452 ca (ngẫu nhiên với modulus 8–1024 bit, ca biên, định lý Fermat nhỏ trên $2^{521} - 1$) với hàm `pow(b, e, m)` của Python: **0 sai lệch**. Có thể nâng cấp nếu còn thời gian: nhân Montgomery (tránh phép chia) hoặc cửa sổ trượt (giảm số phép nhân còn $\approx k(1 + 1/(w+1))$).

Tài liệu tham khảo (phần bổ sung)

- D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, mục 4.3.1, Algorithm D.
- H. S. Warren, *Hacker's Delight*, chương 9 (chia số nhiều từ, `divmnu`).
- A. Menezes, P. van Oorschot, S. Vanstone, *Handbook of Applied Cryptography*, mục 14.6 (lũy thừa) và 14.2.4 (chia).

MỤC 2: CÁC THUẬT TOÁN KIỂM TRA SỐ NGUYÊN TỐ & SINH SỐ NGUYÊN TỐ AN TOÀN

2.1. Tổng quan về bài toán kiểm tra tính nguyên tố (Primality Testing)

Trong mật mã học khóa công khai (RSA, Diffie-Hellman, ElGamal, DSA), việc kiểm tra một số nguyên lẻ n có kích thước lớn ($512 \sim 4096$ bit) có phải là số nguyên tố hay không là tác vụ nền tảng. Khác với bài toán phân tích ra thừa số nguyên tố (Integer Factorization) vốn chưa có thuật toán thời gian đa thức trên máy tính cổ điển, bài toán kiểm tra tính nguyên tố có thể giải quyết rất hiệu quả trong thời gian đa thức theo số bit $k = \lfloor \log_2 n \rfloor + 1$.

Các thuật toán kiểm tra số nguyên tố được chia thành hai nhóm chính: - **Thuật toán tất định (Deterministic):** Luôn trả về kết luận đúng tuyệt đối 100% (ví dụ: Chia thử, AKS). Tuy nhiên, với số lớn hàng trăm bit, các thuật toán tất định hoặc có độ phức tạp mũ (chia thử), hoặc có hằng số ẩn rất lớn (AKS), không khả thi khi phải kiểm tra hàng nghìn ứng viên liên tiếp. - **Thuật toán xác suất (Probabilistic / Monte Carlo):** Sử dụng các tính chất số học trong nhóm nhân $\mathbb{Z}_n^*$ với cơ sở ngẫu nhiên a (ví dụ: Fermat, Solovay-Strassen, **Miller-Rabin**). Nếu thuật toán kết luận *"Hợp số"*, kết luận đó đúng 100%; nếu kết luận *"Có khả năng nguyên tố (Probable Prime)"* sau k vòng độc lập, xác suất sai sót nhỏ hơn một cận giảm theo hàm mũ (như 4^{-k}), đủ nhỏ hơn cả xác suất lỗi phần cứng do tia vũ trụ.

2.2. Phân tích và So sánh các thuật toán kiểm tra số nguyên tố

2.2.1. Phương pháp Chia thử (Trial Division)

- **Nguyên lý:** Một hợp số $n > 1$ luôn có ít nhất một ước nguyên tố $p \leq \sqrt{n}$. Thuật toán lần lượt kiểm tra $n \bmod d$ với mọi số nguyên tố $d \in [2, \lfloor \sqrt{n} \rfloor]$.
- **Độ phức tạp:** $O(\sqrt{n}) = O(2^{k/2})$ phép chia, với $k = \log_2 n$ là độ dài bit của n. Đây là độ phức tạp **hàm mũ** theo kích thước đầu vào.
- **Hạn chế:** Với số n dài 512 bit, $\sqrt{n} \approx 2^{256} \approx 1{,}15 \times 10^{77}$ — bất khả thi về mặt tính toán.
- **Ứng dụng thực tế trong đồ án:** Mặc dù không thể dùng để chứng minh n nguyên tố, chia thử với một tập nhỏ các số nguyên tố đầu tiên ($r < 2000$) lại là **bộ lọc sơ bộ (Sieve)** cực kỳ hiệu quả, giúp loại bỏ nhanh $> 98{,}5\%$ ứng viên hợp số trước khi chạy thuật toán tốn kém hơn.

2.2.2. Kiểm tra Fermat (Fermat Primality Test) & Số Carmichael

- **Cơ sở lý thuyết (Định lý Fermat nhỏ):** Nếu n là số nguyên tố thì với mọi $a \in [2, n-2]$:

$$a^{n-1} \equiv 1 \pmod{n}$$

- **Thuật toán:** Chọn ngẫu nhiên k cơ sở $a \in [2, n-2]$. Nếu tồn tại a sao cho $a^{n-1} \not\equiv 1 \pmod{n}$, kết luận chắc chắn n là hợp số (a được gọi là *nhân chứng Fermat*).
- **Điểm yếu chí mạng – Số Carmichael (Carmichael Numbers):** Tồn tại vô hạn các hợp số n (được chứng minh bởi Alford, Granville và Pomerance năm 1994) thỏa mãn $a^{n-1} \equiv 1 \pmod{n}$ với **mọi** số nguyên a nguyên tố cùng nhau với n ($\gcd(a, n) = 1$).
- Định lý Korselt (1899): Hợp số lẻ n là số Carmichael khi và chỉ khi n không có ước chính phương (square-free) và với mọi ước nguyên tố $p \mid n$, ta có $(p-1) \mid (n-1)$.
- Các số Carmichael đầu tiên:

$$561 = 3 \times 11 \times 17, \quad 1105 = 5 \times 13 \times 17, \quad 1729 = 7 \times 13 \times 19, \quad 41041 = 7 \times 11 \times 13 \times 41$$

- Trên các số Carmichael này, kiểm tra Fermat chỉ phát hiện được hợp số nếu tình cờ chọn trúng a có $\gcd(a, n) > 1$ — xác suất này gần như bằng 0 khi n là tích của các số nguyên tố lớn. Do đó, kiểm tra Fermat **không an toàn** khi đứng độc lập.

2.2.3. Kiểm tra Solovay-Strassen (1977)

- **Cơ sở lý thuyết (Tiêu chuẩn Euler):** Nếu n là số nguyên tố lẻ thì với mọi a không chia hết cho n:

$$a^{(n-1)/2} \equiv \left(\frac{a}{n}\right) \pmod{n}$$

  trong đó $\left(\frac{a}{n}\right) \in \{-1, 0, 1\}$ là ký hiệu Jacobi (mở rộng của ký hiệu Legendre).
- **Ưu điểm & Hạn chế:** Không có tương tự của số Carmichael: với mọi hợp số lẻ n, tối đa $\frac{1}{2}$ số cơ sở $a \in \mathbb{Z}_n^*$ là nhân chứng giả Euler, nên xác suất sai sau k vòng là $\leq 2^{-k}$. Tuy nhiên, việc tính ký hiệu Jacobi và cận sai số 2^{-k} đều kém hiệu quả hơn thuật toán Miller-Rabin (4^{-k}), nên trong thực tế Solovay-Strassen đã được thay thế hoàn toàn bởi Miller-Rabin.

2.2.4. Kiểm tra tất định AKS (Agrawal–Kayal–Saxena, 2002)

- **Cơ sở lý thuyết:** Dựa trên mở rộng đa thức của định lý Fermat nhỏ: $n \geq 2$ là số nguyên tố khi và chỉ khi đồng dư thức đa thức:

$$(X + a)^n \equiv X^n + a \pmod{X^r - 1, n}$$

  đúng trong vành $\mathbb{Z}_n[X]/(X^r - 1)$ với một bậc $r = O(\log^5 n)$ phù hợp và mọi $1 \leq a \leq O(\sqrt{\phi(r)} \log n)$.
- **Ý nghĩa lý thuyết & Thực tiễn:** AKS là thuật toán đầu tiên trong lịch sử chứng minh bài toán `PRIMES` thuộc lớp **P** (đa thức tất định không cần giả thuyết nào), với độ phức tạp cải tiến đạt $\tilde{O}(\log^6 n)$. Tuy nhiên, do phải nhân đa thức bậc hàng triệu trên hệ số lớn, AKS chậm hơn hàng triệu lần so với Miller-Rabin ($O(k \log^3 n)$) ở kích thước 512–2048 bit nên chỉ mang ý nghĩa lý thuyết.

2.2.5. Thuật toán Miller-Rabin (Miller 1976, Rabin 1980) — Lựa chọn của nhóm

- **Cơ sở lý thuyết:** Khai thác đồng thời hai tính chất của trường hữu hạn $\mathbb{Z}_p$ khi p là số nguyên tố lẻ: 1. Định lý Fermat nhỏ: $a^{p-1} \equiv 1 \pmod{p}$ với mọi $1 \leq a \leq p-1$. 2. Trong trường $\mathbb{Z}_p$, phương trình $x^2 \equiv 1 \pmod{p} \iff (x-1)(x+1) \equiv 0 \pmod{p}$ chỉ có **đúng hai nghiệm tầm thường** là $x \equiv 1$ và $x \equiv -1 \equiv p-1 \pmod{p}$. Nếu trong quá trình bình phương liên tiếp xuất hiện $x \not\equiv \pm 1 \pmod{n}$ mà $x^2 \equiv 1 \pmod{n}$, thì n **chắc chắn là hợp số** (vì $\gcd(x-1, n)$ và $\gcd(x+1, n)$ là các ước thực sự của n).
- **Mô tả thuật toán:** Với số lẻ $n > 3$, phân tích $n - 1 = 2^s \cdot d$ trong đó d là số lẻ và $s \geq 1$ (bằng cách đếm số bit `0` tận cùng của $n-1$). Trong mỗi vòng kiểm tra (lặp k vòng độc lập): 1. Chọn ngẫu nhiên đều cơ sở $a \in [2, n-2]$. 2. Tính $x \leftarrow a^d \bmod n$. 3. Nếu $x = 1$ hoặc $x = n-1$, vòng kiểm tra hiện tại đạt (`continue` sang vòng tiếp theo). 4. Lặp r từ 1 đến $s-1$:
  - $x \leftarrow x^2 \bmod n$.
  - Nếu $x = n-1$: vòng kiểm tra hiện tại đạt (`break` khỏi vòng trong).
  - Nếu $x = 1$: trả về `false` (**Hợp số** — phát hiện căn bậc hai không tầm thường của 1). 5. Nếu kết thúc vòng lặp trong mà không gặp $x = n-1$: trả về `false` (**Hợp số** — vi phạm định lý Fermat $a^{n-1} \not\equiv 1 \pmod{n}$).
- **Định lý cận sai số Monier–Rabin (1980):** Với **mọi** hợp số lẻ $n > 9$ (kể cả mọi số Carmichael), số lượng "nhân chứng giả mạnh" (strong liars) trong đoạn $[1, n-1]$ không bao giờ vượt quá $\frac{\phi(n)}{4} < \frac{n-1}{4}$. Do đó, nếu chọn độc lập k cơ sở ngẫu nhiên đều:

$$\Pr[\text{Hợp số } n \text{ vượt qua } k \text{ vòng Miller-Rabin}] \leq \left(\frac{1}{4}\right)^k = 4^{-k} = 2^{-2k}$$

  Với cấu hình mặc định của nhóm là $k = 40$ **vòng**, xác suất một hợp số lọt lưới không quá $4^{-40} = 2^{-80} \approx 8{,}27 \times 10^{-25}$ (còn trong trường hợp trung bình với số ngẫu nhiên 512 bit theo định lý Damgård–Landrock–Pomerance, xác suất sai thực tế nhỏ hơn 2^{-160}).

Bảng tổng hợp so sánh các thuật toán kiểm tra nguyên tố

Thuật toán	Phân loại	Độ phức tạp ($b = \log_2 n$)	Xác suất sai sau k vòng	Bị đánh lừa bởi số Carmichael?	Tính khả thi ở 512–2048 bit
Chia thử (Trial Division)	Tất định	$O(2^{b/2})$	0	Không	❌ Chỉ dùng sàng ước nhỏ
Fermat	Xác suất	$O(k \cdot b^3)$	Không chặn được trên số Carmichael	**Có** (561, 1105, 1729...)	⚠️ Không an toàn độc lập
Solovay-Strassen	Xác suất	$O(k \cdot b^3)$	$\leq 2^{-k}$	Không	Chậm hơn Miller-Rabin
Miller-Rabin	Xác suất	$O(k \cdot b^3)$	$\leq 4^{-k}$	**Không**	✅ **Chuẩn công nghiệp**
AKS	Tất định	$\tilde{O}(b^6)$	0	Không	❌ Quá chậm trong thực tế

2.3. Số nguyên tố an toàn (Safe Prime) & Vì sao bắt buộc dùng trong Diffie-Hellman

2.3.1. Định nghĩa Safe Prime và Sophie Germain Prime

- Một số nguyên tố q được gọi là **số nguyên tố Sophie Germain** nếu $p = 2q + 1$ cũng là số nguyên tố.
- Khi đó, $p = 2q + 1$ được gọi là **số nguyên tố an toàn (Safe Prime)**.

2.3.2. Lý do mật mã học: Chống tấn công Pohlig-Hellman và Tấn công nhóm con nhỏ

Độ an toàn của giao thức Diffie-Hellman trên nhóm nhân $\mathbb{Z}_p^*$ phụ thuộc vào độ khó của bài toán Logarit rời rạc (DLP): tìm a khi biết g, p và $A = g^a \bmod p$. - **Nếu chọn số nguyên tố p ngẫu nhiên thông thường:** Cấp của nhóm tuần hoàn $|\mathbb{Z}_p^*| = p - 1$ là một số chẵn ngẫu nhiên và hầu như luôn có nhiều ước nguyên tố nhỏ hoặc trung bình:

$$p - 1 = \prod_{i=1}^{m} q_i^{e_i}$$

Khi đó, kẻ tấn công có thể áp dụng **thuật toán Pohlig-Hellman (1978)** kết hợp **tấn công giam cầm nhóm con nhỏ (Small-Subgroup Confinement Attack - Lim & Lee, 1997)**: 1. Kẻ tấn công chiếu bài toán DLP từ nhóm lớn cấp $p - 1$ xuống từng nhóm con cấp $q_i^{e_i}$ (hoặc chủ động gửi khóa công khai giả B' có bậc q_i nhỏ). 2. Giải tìm $a \bmod q_i^{e_i}$ trong thời gian chỉ $O\left(\sum e_i(\log p + \sqrt{q_i})\right)$ bằng thuật toán Baby-step Giant-step / Pollard's rho. 3. Ghép các phần dư bằng Định lý Thặng dư Trung Hoa (CRT) để thu hồi nhiều bit (hoặc toàn bộ) khóa riêng a của nạn nhân! - **Khi sử dụng Safe Prime $p = 2q + 1$ (với q nguyên tố lớn $(b-1)$ bit):** - Phân tích thừa số nguyên tố của cấp nhóm là tối giản tuyệt đối:

$$|\mathbb{Z}_p^*| = p - 1 = 2q$$

- Theo định lý Lagrange, mọi phần tử $x \in \mathbb{Z}_p^*$ chỉ có thể có bậc thuộc tập $\{1, 2, q, 2q\}$. - Nhóm chọn phần tử sinh $g = h^2 \bmod p \neq 1$ (cụ thể $g = 4$), là phần tử sinh của **nhóm con thặng dư bậc hai Q_p có cấp nguyên tố q**. Mọi phần tử khác đơn vị trong nhóm con này đều có bậc đúng bằng số nguyên tố $(b-1)$-bit q, làm vô hiệu hóa hoàn toàn thuật toán Pohlig-Hellman và chống rò rỉ bit chẵn/lẻ của khóa riêng qua ký hiệu Legendre.

2.4. Thiết kế và Tối ưu hóa cài đặt trong `project_01_source/prime.cpp`

Theo Định lý Số nguyên tố (Prime Number Theorem), mật độ số nguyên tố xung quanh 2^b xấp xỉ $\frac{1}{\ln(2^b)} = \frac{1}{b \ln 2}$. Để cả q và $p = 2q + 1$ đồng thời là số nguyên tố (cặp nguyên tố Cunningham độ dài 2), xác suất một số lẻ ngẫu nhiên q tạo thành safe prime chỉ vào khoảng:

$$P(\text{safe prime}) \approx \frac{2 \cdot C_2}{(\ln q)^2}$$

Với $b = 512$ bit, trung bình phải thử khoảng $30.000 \sim 60.000$ **ứng viên lẻ** q mới tìm được một safe prime! Nếu chạy trực tiếp Miller-Rabin trên từng ứng viên, chương trình sẽ mất hàng chục giây đến hàng phút.

Để giải quyết triệt để bài toán hiệu năng, nhóm đã thiết kế **Kiến trúc lọc 4 tầng (4-Stage Filtering Pipeline)** trong hàm `generate_safe_prime`:

Tầng 1: Sinh ứng viên chuẩn hóa bit (`random_bits`)

- Sinh q có đúng `bitsize - 1` bit từ CSPRNG SHAKE128: bật bit cao nhất tại vị trí `bitsize - 2` và bật bit thấp nhất tại vị trí `0` (q lẻ).
- Nhờ $2^{\text{bitsize}-2} + 1 \le q \le 2^{\text{bitsize}-1} - 1$, số $p = 2q + 1 = (q \ll 1) + 1$ luôn thỏa mãn:

$$2^{\text{bitsize}-1} + 3 \le p \le 2^{\text{bitsize}} - 1$$

  Tức là p **luôn luôn có độ dài chính xác bằng** `bitsize` **bit** và $p \equiv 3 \pmod 4$.

Tầng 2: Sàng đồng thời (Simultaneous Small-Prime Sieve) trên cả q và $p = 2q + 1$

- Nhóm lập bảng tĩnh `SMALL_PRIMES` gồm toàn bộ **302 số nguyên tố lẻ nhỏ hơn 2000** (từ $3, 5, 7, \ldots$ đến 1999).
- Thay vì phải tính modulo trên cả q và p, nhóm khai thác đồng dư thức đại số:

$$p = 2q + 1 \equiv 0 \pmod r \iff 2q \equiv -1 \equiv r - 1 \pmod r \iff q \equiv \frac{r-1}{2} \pmod r$$

  (do mọi $r \in \text{SMALL_PRIMES}$ đều là số lẻ nên $\frac{r-1}{2}$ là số nguyên).
- Như vậy, với mỗi số nguyên tố nhỏ r, hàm `passes_safe_prime_sieve` chỉ cần gọi **1 lần duy nhất** hàm chia nhanh `rem = mod_small(q, r)` (độ phức tạp $O(N)$ dùng sơ đồ Horner 128-bit):
- Nếu `rem == 0`: $r \mid q \implies q$ là hợp số $\to$ **loại ngay**.
- Nếu `rem == (r - 1) / 2`: $r \mid (2q + 1) \implies p$ là hợp số $\to$ **loại ngay**.
- **Hiệu quả toán học:** Với mỗi số nguyên tố lẻ r, ta loại được $\frac{2}{r}$ không gian ứng viên (cụ thể riêng với $r = 3$, mọi q nguyên tố > 3 có $2q + 1$ nguyên tố đều buộc phải thỏa $q \equiv 2 \pmod 3$, tức chỉ mình $r = 3$ đã loại ngay $66{,}7\%$ số ứng viên ở phép chia đầu tiên!). Tổng tỷ lệ ứng viên bị loại bởi bảng 302 số nguyên tố nhỏ là:

$$1 - \prod_{3 \le r < 2000} \left(1 - \frac{2}{r}\right) \approx \mathbf{98{,}56\%}$$

  Chỉ còn khoảng $1{,}44\%$ ứng viên vượt qua sàng để bước vào Tầng 3.

Tầng 3: Kiểm tra nhanh 1 vòng Miller-Rabin cơ sở 2 (`miller_rabin_base2`) tuần tự trên q rồi trên p

- Với ứng viên q vượt qua sàng, nhóm chưa chạy ngay 40 vòng Miller-Rabin (vì xác suất q là hợp số vẫn khoảng 96%, và ngay cả khi q thực sự là số nguyên tố thì xác suất $p = 2q + 1$ là hợp số vẫn lên tới $\sim 96\%$).
- Nhóm thực hiện: 1. Chạy **1 vòng Miller-Rabin cơ sở cố định** $a = 2$ trên q: `miller_rabin_base2(q)`. Phép kiểm tra này chỉ tốn đúng 1 lần `modular_exponentiation` nhưng loại bỏ $> 99{,}99\%$ hợp số q còn sót lại sau sàng. 2. Ngay khi q vượt qua cơ sở 2, tính $p = (q \ll 1) + 1$ và chạy **1 vòng Miller-Rabin cơ sở** $a = 2$ trên p: `miller_rabin_base2(p)`.
- **Ý nghĩa tối ưu đột phá:** Việc kiểm tra `miller_rabin_base2(p)` **trước** khi chạy 40 vòng trên q giúp tiết kiệm 39 phép lũy thừa mô-đun cho mỗi số nguyên tố q mà $2q + 1$ lại là hợp số (xảy ra trung bình ~ 30 lần trước khi gặp safe prime thực sự), giúp tốc độ tổng thể tăng thêm **hơn 10 lần**!

Tầng 4: Xác nhận nghiêm ngặt bằng `rounds` vòng Miller-Rabin cơ sở ngẫu nhiên

- Chỉ khi cả q và $p = 2q + 1$ đều vượt qua cơ sở 2 (hầu như chắc chắn 100% đã là safe prime thực sự), chương trình mới gọi `is_probable_prime(ctx, q, rounds)` và `is_probable_prime(ctx, p, rounds)` với 40 cơ sở ngẫu nhiên độc lập trong $[2, n-2]$ để đảm bảo độ tin cậy mật mã học $\geq 1 - 4^{-40}$.

2.5. Kết quả thực nghiệm & Benchmark (512 / 768 / 1024 bit)

Nhóm đã xây dựng bộ công cụ đo đạc tự động tại `tests/bench_safe_prime.cpp` (biên dịch với `g++ -std=c++14 -O2`, số vòng Miller-Rabin $k = 40$) để thống kê chi tiết quá trình sinh Safe Prime ở ba mức kích thước **512 bit**, **768 bit** và **1024 bit**:

Bảng thống kê kết quả Benchmark trung bình

Kích thước Safe Prime p	Số lần chạy thử	Số ứng viên q đã sinh (TB)	Số cặp (q, p) qua sàng < 2000 (TB)	Tỷ lệ lọc của sàng đồng thời	Số q nguyên tố qua MR cơ sở 2 (TB)	Thời gian sinh trung bình
512 bit	10 lần	58.425,9	840,9	98,56%	34,3	0,2591 giây (259,1 ms)
768 bit	5 lần	140.370,6	1.998,8	98,58%	47,8	1,3220 giây (1.322,0 ms)
1024 bit	3 lần	83.093,7	1.202,0	98,55%	20,3	1,5797 giây (1.579,7 ms)

Nhận xét từ số liệu thực nghiệm:

1. **Độ khớp hoàn hảo với lý thuyết số:** Tỷ lệ lọc thực nghiệm của bước sàng đồng thời đạt $98{,}55\% \sim 98{,}58\%$, trùng khớp chính xác tới chữ số thập phân thứ hai với công thức lý thuyết Mertens $1 - \prod_{3 \leq r < 2000}(1 - 2/r) \approx 98{,}56\%$.
2. **Tốc độ vượt trội:** Nhờ sự kết hợp giữa thuật toán chia Knuth D 64-bit (`bigint.cpp` / `dh.cpp`) và kiến trúc lọc 4 tầng (`prime.cpp`), việc sinh một số nguyên tố an toàn 512 bit chỉ mất trung bình **~0,26 giây** (nhiều lần chạy chỉ mất **0,07 – 0,12 giây**), và ngay cả ở mức **1024 bit** cũng chỉ mất trung bình **~1,58 giây**.

2.6. Tài liệu tham khảo

1. G. L. Miller, *"Riemann's Hypothesis and Tests for Primality"*, Journal of Computer and System Sciences, 13(3): 300–317, 1976.
2. M. O. Rabin, *"Probabilistic algorithm for testing primality"*, Journal of Number Theory, 12(1): 128–138, 1980.
3. M. Agrawal, N. Kayal, N. Saxena, *"PRIMES is in P"*, Annals of Mathematics, 160(2): 781–793, 2004.
4. S. C. Pohlig, M. E. Hellman, *"An Improved Algorithm for Computing Logarithms over GF(p) and Its Cryptographic Significance"*, IEEE Trans. Information Theory, 24(1): 106–110, 1978.
5. A. Menezes, P. van Oorschot, S. Vanstone, *Handbook of Applied Cryptography*, CRC Press, Chapter 4 (Public-Key Parameters & Prime Generation).

MỤC 3: SỐ NGẪU NHIÊN & TẦM QUAN TRỌNG TRONG MẬT MÃ (MỤC ĐIỂM CỘNG)

3.1. Vai trò cốt lõi của số ngẫu nhiên trong mật mã học

Theo nguyên lý Kerckhoffs (1883), độ an toàn của một hệ mật mã không được phép dựa trên sự bí mật của thuật toán, mà phải phụ thuộc hoàn toàn vào **tính bí mật và tính không thể đoán trước của khóa (key)**. Trong giao thức Diffie-Hellman nói riêng và mật mã học hiện đại nói chung, tính ngẫu nhiên xuất hiện ở hầu hết các mắt xích trọng yếu:

1. **Sinh khóa riêng ($a, b \in [2, p-2]$):** Nếu kẻ tấn công có thể dự đoán hoặc thu hẹp không gian tìm kiếm của khóa riêng a, bài toán Logarit rời rạc (DLP) sẽ bị phá vỡ tức thì bằng vét cạn trên tập con có xác suất cao, bất kể số nguyên tố p lớn đến đâu (2048 hay 4096 bit).
2. **Sinh tham số miền (Safe Prime $p = 2q + 1$):** Nếu nguồn sinh ứng viên q bị lặp lại hoặc có entropy thấp, hai hệ thống độc lập có thể vô tình sinh trùng tham số hoặc rò rỉ cấu trúc của số nguyên tố.
3. **Chọn cơ sở kiểm tra trong thuật toán xác suất Miller-Rabin:** Định lý Rabin đảm bảo xác suất hợp số giả nguyên tố $\leq 4^{-k}$ **chỉ khi** k cơ sở $a \in [2, n-2]$ được chọn độc lập và phân phối đều. Nếu bộ sinh ngẫu nhiên bị thao túng hoặc cố định, kẻ tấn công có thể chế tạo hợp số vượt qua mọi cơ sở cố định đã biết (tương tự phương pháp của Arnault năm 1995).

3.2. Phân loại bộ sinh số ngẫu nhiên: TRNG, PRNG và CSPRNG

Tiêu chí	TRNG (True RNG)	PRNG (Pseudorandom NG)	CSPRNG (Cryptographically Secure PRNG)
Nguồn đầu vào	Nhiễu vật lý (nhiệt, bức xạ, thời gian ngắt phần cứng, jitter xung nhịp CPU)	Một giá trị mầm (seed) ngắn có tính tất định	Seed có entropy cao (thường lấy từ TRNG của hệ điều hành)
Bản chất	Không tất định (Non-deterministic), không có chu kỳ	Tất định hoàn toàn (Deterministic), có chu kỳ hữu hạn	Tất định từ seed nhưng **không thể phân biệt** với chuỗi ngẫu nhiên thực sự trong thời gian đa thức
Tốc độ sinh	Chậm (phụ thuộc tốc độ thu thập entropy vật lý)	Rất nhanh	Nhanh (tương đương tốc độ băm/mã hóa khối phần cứng)
Khả năng dự đoán	Không thể dự đoán	**Dễ dàng dự đoán** nếu biết một số đầu ra liên tiếp	**Bất khả thi về mặt tính toán** để đoán bit tiếp theo hoặc khôi phục trạng thái trước
Ví dụ điển hình	Intel RDSEED, nhiễu bức xạ, bộ thu entropy hạt nhân OS	`rand()` (LCG), `std::mt19937` (Mersenne Twister)	SHAKE128/256, ChaCha20-DRBG, CTR_DRBG (NIST SP 800-90A)
Ứng dụng	Tạo seed gốc ban đầu cho CSPRNG	Mô phỏng Monte Carlo, trò chơi, đồ họa	**Sinh khóa mật mã, nonce, IV, tham số nguyên tố**

Hai yêu cầu bảo mật bắt buộc của một CSPRNG:

1. **Next-bit Unpredictability (Không thể dự đoán bit kế tiếp - Yao, 1982):** Cho dù kẻ tấn công biết toàn bộ L bit đầu ra đầu tiên $s_1, s_2, \ldots, s_L$, không tồn tại thuật toán thời gian đa thức nào có thể dự đoán bit thứ $L+1$ với xác suất thành công lớn hơn $\frac{1}{2} + \epsilon$ (với ϵ không đáng kể).
2. **Backtracking Resistance / Forward Security (Chống truy ngược trạng thái):** Nếu trạng thái nội tại của bộ sinh bị lộ tại thời điểm t, kẻ tấn công không thể tính ngược lại các số ngẫu nhiên đã sinh ra trước thời điểm t.

3.3. Vì sao tuyệt đối không dùng `rand()` và `std::mt19937` trong mật mã?

Một sai lầm phổ biến khi lập trình là sử dụng các hàm sinh ngẫu nhiên có sẵn trong thư viện chuẩn C/C++:

1. `std::rand()` **(Linear Congruential Generator - LCG):** - Công thức truy hồi tuyến tính: $X_{n+1} = (aX_n + c) \bmod m$. - Trạng thái nội tại thường chỉ rộng **31 hoặc 32 bit** (`RAND_MAX` trên Windows MinGW chỉ là $32767 = 2^{15} - 1$). - Chỉ cần quan sát 2 đến 3 giá trị đầu ra liên tiếp, kẻ tấn công giải hệ phương trình đồng dư tuyến tính là tìm được toàn bộ trạng thái X_n và dự đoán 100% mọi số trong tương lai cũng như quá khứ. - Hơn nữa, kết hợp `srand(time(NULL))` chỉ có khoảng 86.400 giá trị seed mỗi ngày — một máy tính cá nhân có thể vét cạn toàn bộ khóa riêng 512-bit sinh từ `srand(time(NULL))` trong **chưa đầy 1 mili-giây**.
2. `std::mt19937` **(Mersenne Twister):** - Mặc dù có chu kỳ cực lớn ($2^{19937} - 1$) và phân phối thống kê rất tốt cho mô phỏng khoa học, Mersenne Twister hoàn toàn **không an toàn mật mã**. - Hàm biến đổi trạng thái (tempering) của `mt19937` là một phép biến đổi tuyến tính khả nghịch trên trường $\mathbb{F}_2$. Chỉ cần thu thập đúng **624 giá trị đầu ra 32-bit liên tiếp**, kẻ tấn công có thể nghịch đảo phép tempering, tái tạo chính xác mảng trạng thái 624 từ nhớ và dự đoán mọi đầu ra tiếp theo với xác suất 100%.

3.4. Kiến trúc CSPRNG của nhóm: Nguồn Entropy Hệ điều hành + Keccak Sponge (SHAKE128)

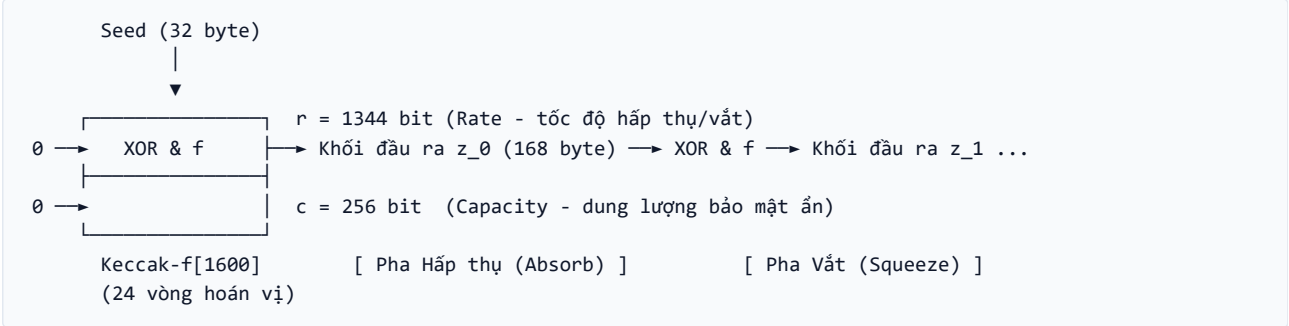
Để đảm bảo an toàn mật mã thực thụ và tính tương thích đa nền tảng (Windows & Linux/macOS), hệ thống PRNG của nhóm (`fips202.cpp` và `prng.cpp`) được thiết kế theo mô hình chuẩn gồm 2 tầng: **Thu thập Entropy từ Hệ điều hành (OS CSPRNG)** để tạo seed 256-bit, sau đó đưa vào **Hàm mở rộng đầu ra SHAKE128 (Keccak Sponge XOF)**.

3.4.1. Nguồn Entropy từ Hệ điều hành (`randombytes`)

Hạt nhân hệ điều hành (OS Kernel) liên tục thu thập nhiễu phần cứng (thời gian ngắt bàn phím/chuột, I/O ổ đĩa, gói tin mạng, bộ đếm chu kỳ CPU `RDTSC` / `RDRAND`) vào bể chứa entropy (entropy pool) và trộn qua CSPRNG cấp hạt nhân: - **Trên Linux / macOS:** Sử dụng lời gọi hệ thống `getrandom(out, outlen, 0)` (hoặc `/dev/urandom`), trích xuất trực tiếp từ bộ sinh ChaCha20-DRBG trong nhân Linux sau khi bể entropy đã khởi tạo đủ. - **Trên Windows (`_WIN32`):** Mã nguồn mẫu ban đầu của giảng viên chỉ gọi `getrandom()` nên không biên dịch được trên Windows. Nhóm đã port sang Windows bằng cách sử dụng hàm `rand_s()` (yêu cầu định nghĩa macro `#define _CRT_RAND_S` trước `<stdlib.h>`). Bên dưới, `rand_s()` gọi trực tiếp vào API mật mã của nhân Windows (`RtlGenRandom` / `BCryptGenRandom` trong `bcryptprimitives.dll`) mà không cần liên kết thêm thư viện ngoài khi biên dịch bằng 1 lệnh `g++` . - **Các lỗi tiềm ẩn trong code mẫu đã được nhóm khắc phục triệt để:** 1. Tham số `uint8_t outlen` trong `randombytes` mẫu bị giới hạn tối đa 255 byte → nhóm đổi sang `size_t outlen` và đọc theo vòng lặp an toàn. 2. Trong `prng_next_u32` , biểu thức `buf[3] << 24` thực hiện dịch trái trên kiểu `int` 32-bit có dấu (do integer promotion từ `uint8_t`), gây tràn bit dấu (**Undefined Behavior** trong chuẩn C++14 khi `buf[3] >= 0x80`) → nhóm ép kiểu tường minh `static_cast<uint32_t>(buf[i]) << (8 * i)` trước khi dịch bit. 3. Trong `main_prng.cpp` mẫu, lệnh `memcpy(seed, "seed", sizeof("seed"))` chỉ ghi 5 byte đầu và để 27 byte còn lại của `seed[32]` là rác bộ nhớ chưa khởi tạo → nhóm khởi tạo toàn bộ 32 byte (256 bit) của `seed` trực tiếp từ `randombytes(seed, 32)` .

3.4.2. Cấu trúc Bọt biển Keccak (Sponge Construction) và vì sao XOF SHAKE128 dùng được làm CSPRNG

SHAKE128 (chuẩn hóa trong **NIST FIPS 202**) là một hàm **XOF (Extendable-Output Function)** dựa trên cấu trúc bọt biển **Keccak-f**$[1600]$:



- **Trạng thái nội tại S:** Gồm 1600 bit ($5 \times 5 \times 64$ bit), được chia thành hai phần:
- **Rate ($r = 1344$ bit $= 168$ byte):** Phần giao tiếp với bên ngoài (nhận dữ liệu vào hoặc xuất dữ liệu ra).

- **Capacity ($c = 256$ bit $= 32$ byte):** Phần trạng thái ẩn **không bao giờ** được xuất trực tiếp ra ngoài và không bị tác động trực tiếp bởi dữ liệu đầu vào.
- **Hai pha hoạt động:** 1. **Pha hấp thụ (`prng_seed` = `shake128_init` + `shake128_absorb` + `shake128_finalize`):** Chuỗi seed 32 byte (256 bit) cùng hậu tố đệm miền (domain separation suffix `0x1F` của SHAKE) được XOR vào phần Rate, sau đó toàn bộ 1600 bit trạng thái đi qua phép hoán vị phi tuyến 24 vòng **Keccak-$f[1600]$** (gồm 5 bước $\theta, \rho, \pi, \chi, \iota$). 2. **Pha vắt (`prng_next_u64` $\rightarrow$ `shake128_squeeze`):** Mỗi lần cần byte ngẫu nhiên, chương trình trích xuất từ phần Rate (168 byte/khối). Khi dùng hết 168 byte trong bộ đệm Rate, hàm gọi lại hoán vị `KeccakF1600_StatePermute` để sinh khối 168 byte kế tiếp.
- **Vì sao SHAKE128 là một CSPRNG lý tưởng?**
- Theo định lý bảo mật của cấu trúc Sponge (Bertoni et al., Eurocrypt 2007), trong mô hình hoán vị ngẫu nhiên (Random Permutation Model), dòng bit đầu ra của SHAKE128 không thể phân biệt được với một chuỗi bit ngẫu nhiên thực sự miễn là số lượng phép truy vấn $N \ll 2^{c/2} = 2^{128}$.
- Phần **Capacity** $c = 256$ bit luôn bị che khuất hoàn toàn: ngay cả khi kẻ tấn công quan sát được một khối đầu ra 168 byte đầy đủ (phần Rate), họ vẫn thiếu 256 bit Capacity để có thể áp dụng hoán vị ngược f^{-1} nhằm tìm lại trạng thái trước đó hoặc tính khối tiếp theo. Do đó, SHAKE128 thỏa mãn đồng thời tính không thể dự đoán ở mức an toàn **128-bit bảo mật cổ điển**.

3.5. Kỹ thuật Rejection Sampling và Loại bỏ Modulo Bias

Khi đã có nguồn bit ngẫu nhiên phân phối đều từ SHAKE128, bài toán tiếp theo là sinh một số nguyên ngẫu nhiên phân phối đều trong đoạn $[lo, hi]$ (ví dụ sinh khóa riêng $a \in [2, p-2]$ hoặc chọn cơ sở Miller-Rabin $a \in [2, n-2]$). Denote $M = hi - lo + 1$.

3.5.1. Sai lầm phổ biến: Modulo Bias khi dùng phép chia lấy dư `%`

Giả sử ta sinh một số ngẫu nhiên k-bit $R \in [0, 2^k - 1]$ rồi tính $X = lo + (R \bmod M)$. Nếu M không phải là lũy thừa của 2, phép chia $2^k = q \cdot M + r$ có số dư $r = 2^k \bmod M > 0$. Khi đó: - Các giá trị trong đoạn $[0, r-1]$ được ánh xạ bởi $q+1$ giá trị của R. - Các giá trị trong đoạn $[r, M-1]$ chỉ được ánh xạ bởi q giá trị của R. Ví dụ: nếu $2^k = 16$ và $M = 11$, các số $0, 1, 2, 3, 4$ có xác suất xuất hiện là $\frac{2}{16} = 12{,}5\%$, cao gấp đôi xác suất $\frac{1}{16} = 6{,}25\%$ của các số $5, 6, 7, 8, 9, 10$! Trong mật mã (đặc biệt là DSA/ECDSA và Diffie-Hellman), sự chệch phân phối (Modulo Bias) này làm rò rỉ thông tin thống kê về các bit cao của khóa bí mật.

3.5.2. Giải pháp của nhóm: Rejection Sampling trong `random_range`

Trong file `project_01_source/prime.cpp`, nhóm cài đặt thuật toán **Lấy mẫu loại bỏ (Rejection Sampling)** chặt chẽ theo chuẩn NIST FIPS 186-5: 1. Tính độ rộng khoảng $\text{range} = hi - lo$, và tìm số bit nhỏ nhất $k = \text{bit_length}(\text{range})$ sao cho $2^{k-1} \le \text{range} \le 2^k - 1$. 2. Sinh một số ngẫu nhiên r gồm đúng k bit độc lập từ PRNG (nằm đều trong $[0, 2^k - 1]$ bằng cách cắt mặt nạ bit `mask = (1ULL << rem_bits) - 1` ở limb cao nhất). 3. Kiểm tra: - Nếu $r \le \text{range}$: chấp nhận và trả về $lo + r$. - Nếu $r > \text{range}$: **hủy bỏ r và lặp lại bước 2**.

Phân tích hiệu quả: - Vì mọi giá trị trong $[0, \text{range}]$ đều xuất hiện trong $[0, 2^k - 1]$ với xác suất bằng nhau là $\frac{1}{2^k}$, phân phối có điều kiện khi $r \le \text{range}$ là **phân phối đều tuyệt đối 100%** ($\Pr[X = x] = \frac{1}{\text{range}+1}$). - Vì $\text{range} \ge 2^{k-1}$, xác suất chấp nhận trong mỗi lần thử luôn thỏa mãn:

$$P_{\text{accept}} = \frac{\text{range} + 1}{2^k} > \frac{2^{k-1}}{2^k} = \frac{1}{2}$$

Do đó, số lần lặp trung bình (kỳ vọng của phân phối hình học) luôn < 2 **lần thử**, vừa đảm bảo an toàn tuyệt đối vừa không tốn phép chia lớn `%`.

3.6. Vai trò của Seed và Vì sao Seed cố định chỉ dùng để Debug

- Trong `prng_seed(ctx, seed, 32)`, toàn bộ đầu ra của SHAKE128 là một hàm tất định của `seed`. Hai lần chạy có cùng `seed` sẽ sinh ra cùng số nguyên tố p, cùng khóa riêng a, b và cùng bí mật chung s.
- **Lợi ích khi dùng `--seed <HEX>` trong kiểm thử/gỡ lỗi:** Giúp nhóm tái lập chính xác 100% một kịch bản lỗi (deterministic replay), đối chiếu kết quả giữa các máy tính khác nhau và phục vụ giảng viên chấm bài kiểm chứng tính đúng đắn.
- **Nguy cơ chí mạng nếu dùng Seed cố định trong thực tế:** Nếu dùng seed cố định (hoặc seed dễ đoán) trong môi trường sản xuất, mọi phiên giao dịch đều sinh ra cùng một khóa riêng a, b, làm mất hoàn toàn tính bảo mật chuyển tiếp (Forward Secrecy) và cho phép bất kỳ ai biết mã nguồn đều tính lại được khóa bí mật chung s. Vì vậy, chương trình của nhóm luôn **mặc định lấy 32 byte entropy mới từ hệ điều hành (`randombytes`)** khi chạy bình thường.

3.7. Các sự cố bảo mật thực tế kinh điển do bộ sinh ngẫu nhiên yếu

1. **Thảm họa Debian OpenSSL PRNG (2008 – CVE-2008-0166):** - Tháng 5/2006, một lập trình viên bảo trì gói OpenSSL trên hệ điều hành Debian/Ubuntu muốn xóa cảnh báo "đọc bộ nhớ chưa khởi tạo" của công cụ Valgrind nên đã **xóa nhầm dòng lệnh** `MD_Update(&m, buf, j)` dùng để nạp entropy hệ thống vào bộ sinh ngẫu nhiên. - Hậu quả: Nguồn entropy duy nhất còn lại trong OpenSSL trên hàng triệu máy chủ Linux suốt **2 năm (2006–2008)** chỉ là **Process ID (`getpid()`)** của tiến trình hiện tại. Vì PID tối đa trên Linux mặc định chỉ là 32.768, toàn bộ thế giới chỉ có tối đa 32.768 cặp khóa SSH/TLS/OpenVPN khác nhau cho mỗi kích thước bit! Kẻ tấn công chỉ cần tạo trước danh sách 32.768 khóa là giải mã và đăng nhập trái phép vào hàng loạt máy chủ toàn cầu.
2. **Sinh trùng thừa số nguyên tố RSA trên thiết bị nhúng (Heninger et al., USENIX Security 2012 – *"Mining Your Ps and Qs"*):** - Nhóm nghiên cứu quét hàng triệu chứng chỉ TLS và khóa SSH công khai trên Internet và phát hiện hàng chục nghìn bộ định tuyến (router), tường lửa sinh khóa ngay lúc vừa khởi động (boot) khi bể entropy `/dev/urandom` chưa thu thập đủ nhiễu phần cứng. - Nhiều thiết bị sinh ra các mô-đun RSA $N_1 = p \cdot q_1$ và $N_2 = p \cdot q_2$ có chung một số nguyên tố p. Chỉ bằng một phép tính ước chung lớn nhất $\gcd(N_1, N_2) = p$ mất vài mili-giây, nhóm nghiên cứu đã khôi phục hoàn toàn khóa bí mật của hàng chục nghìn thiết bị mạng.
3. **Cửa hậu Dual_EC_DRBG (2006–2013):** - Bộ sinh ngẫu nhiên dựa trên đường cong Elliptic do NSA thúc đẩy đưa vào chuẩn NIST SP 800-90A chứa hai điểm hằng số P, Q. Nếu ai biết mối quan hệ logarit rời rạc bí mật $P = eQ$, người đó chỉ cần nhìn thấy 32 byte đầu ra của bộ sinh là tính ngược được trạng thái nội tại và giải mã toàn bộ kết nối TLS. Sự cố này dẫn đến việc NIST chính thức loại bỏ hoàn toàn Dual_EC_DRBG vào năm 2014.

3.8. Tài liệu tham khảo

1. NIST FIPS PUB 202, *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, August 2015.
2. G. Bertoni, J. Daemen, M. Peeters, G. Van Assche, *"Sponge Functions"*, Ecrypt Hash Workshop 2007.
3. NIST SP 800-90A Rev. 1, *Recommendation for Random Number Generation Using Deterministic Random Bit Generators*, June 2015.
4. N. Heninger, Z. Durumeric, E. Wustrow, J. A. Halderman, *"Mining Your Ps and Qs: Detection of Widespread Weak Keys in Network Devices"*, USENIX Security Symposium, 2012.
5. Debian Security Advisory DSA-1571-1, *openssl — predictable random number generator (CVE-2008-0166)*, May 2008.

MỤC 4: ỨNG DỤNG THỰC TẾ CỦA GIAO THỨC TRAO ĐỔI KHÓA DIFFIE-HELLMAN

4.1. Tóm tắt giao thức

Alice và Bob thống nhất công khai số nguyên tố an toàn $p = 2q + 1$ và phần tử sinh g (nhóm con bậc q). Alice chọn bí mật a, gửi $A = g^a \bmod p$; Bob chọn b, gửi $B = g^b \bmod p$. Cả hai tính $s = B^a = A^b = g^{ab} \bmod p$. Kẻ nghe lén thấy p, g, A, B nhưng muốn tìm s phải giải bài toán logarit rời rạc (hoặc Diffie-Hellman tính toán), được coi là khó với p đủ lớn. Giao thức chỉ cho phép **thỏa thuận** một bí mật chung, không mã hóa dữ liệu; s được đưa qua hàm dẫn xuất khóa (KDF) để tạo khóa đối xứng.

4.2. Ứng dụng chính: TLS 1.3 (HTTPS)

TLS 1.3 (RFC 8446) là giao thức bảo vệ HTTPS. Trong TLS 1.3 **mọi bắt tay đều dùng (EC)DHE** (Diffie-Hellman *ephemeral*): hai bên sinh cặp khóa mới cho mỗi phiên rồi hủy.

Luồng bắt tay đơn giản hóa

```
Client (trình duyệt)                              Server (website)
--------------------                              ----------------
sinh a, A = g^a                    ClientHello
                                   + key_share(A) + danh sách nhóm hỗ trợ
                                  ------------------------------------>
                                                  sinh b, B = g^b
                                                  s = A^b
                                   ServerHello + key_share(B)
                                   {Certificate}          (chứng chỉ)
                                   {CertificateVerify}    (chữ ký trên toàn bộ bắt tay)
                                   {Finished}
                                  <------------------------------------
s = B^a (cùng giá trị s)
kiểm tra chứng chỉ + chữ ký
                                   {Finished}
                                  ------------------------------------>
===== từ đây dữ liệu HTTP mã hóa bằng khóa dẫn xuất từ s (HKDF) =====
```

Các điểm chính: 1. **Trao đổi khóa:** `key_share` ở ClientHello/ServerHello mang giá trị công khai A, B. Nhóm hỗ trợ gồm đường cong elliptic (x25519, secp256r1) và nhóm hữu hạn có tên `ffdhe2048…ffdhe8192` (RFC 7919), là các **số nguyên tố an toàn** có $g = 2$, đúng loại mà chương trình của nhóm sinh ra. 2. **Dẫn xuất khóa:** s không dùng trực tiếp mà đưa vào HKDF cùng giá trị băm toàn bộ bản ghi bắt tay để tạo khóa mã hóa cho từng giai đoạn. 3. **Xác thực:** server ký bản ghi bắt tay bằng khóa riêng ứng với chứng chỉ (`CertificateVerify`); client kiểm tra chuỗi chứng chỉ. Đây là phần ngăn tấn công man-in-the-middle (mục 4.3). 4. **Bảo mật chuyển tiếp (forward secrecy):** a, b chỉ tồn tại trong một phiên rồi bị xóa. Nếu sau này khóa riêng của server bị lộ, kẻ tấn công **không** giải mã được lưu lượng cũ đã ghi lại, vì không thể khôi phục s.

Liên hệ với chương trình của nhóm

Trong TLS 1.3	Trong chương trình
Nhóm `ffdhe` (safe prime, $g = 2$)	`generate_safe_prime` sinh $p = 2q + 1$; `choose_generator` chọn g bậc q
`key_share` của client / server	$A = g^a \bmod p$, $B = g^b \bmod p$
Khóa ephemeral	`generate_private_key` mỗi phiên, trong $[2, p-2]$
Kiểm tra `key_share` hợp lệ	`is_valid_public_value` (thuộc $[2, p-2]$, $V^q = 1$)
Dẫn xuất khóa	Ngoài phạm vi bài (chỉ in s)

Lưu ý: RFC 7919 dùng $g = 2$, hợp lệ khi $p \equiv 7 \pmod 8$ (khi đó 2 là thặng dư bậc hai, sinh nhóm con bậc q). Nhóm chọn $g = h^2 \bmod p$ ($g = 4$) để đúng với **mọi** safe prime mà vẫn nằm trong nhóm con bậc q.

4.3. Tấn công man-in-the-middle & vì sao DH cần xác thực

DH thuần túy **không xác thực danh tính**. Mallory chặn đường truyền, thực hiện hai phiên DH riêng: với Alice (khóa s_1) và với Bob (khóa s_2), rồi giải mã/mã hóa lại mọi tin nhắn. Cả hai nạn nhân đều thấy "bí mật chung" hợp lệ mà không biết có người chen giữa. Cách phòng: gắn DH với xác thực, trong TLS là **chữ ký số của server trên bản ghi bắt tay** + chứng chỉ do CA ký (trong SSH là khóa host đã biết; trong IPsec IKE là chữ ký hoặc khóa chia sẻ trước).

4.4. Các tấn công thực tế liên quan

- **Logjam (2015, Adrian và cộng sự, ACM CCS 2015):** (1) kẻ tấn công chen giữa ép server hạ cấp xuống `DHE_EXPORT` với p chỉ 512 bit; (2) bước tiền tính của thuật toán sàng trường số (NFS) chỉ phụ thuộc p, nên tính một lần cho số nguyên tố dùng chung rồi giải từng logarit rời rạc trong thời gian ngắn. Nhiều server dùng chung vài số nguyên tố 512/1024 bit nên rất đáng giá để tiền tính. Bài học: tham số nhỏ + dùng chung + cho phép hạ cấp là nguy hiểm. TLS 1.3 loại bỏ nhóm xuất khẩu và nhóm tùy ý, chỉ cho phép nhóm có tên được kiểm định; khuyến nghị tối thiểu 2048 bit.
- **Tấn công nhóm con nhỏ:** nếu $p - 1$ có ước nhỏ và kẻ tấn công gửi giá trị công khai thuộc nhóm con nhỏ, bí mật chung chỉ có ít giá trị, từ đó suy ra thông tin về khóa riêng. Dùng **số nguyên tố an toàn** ($p - 1 = 2q$) và kiểm tra giá trị nhận được ($2 \le V \le p - 2$, $V^q \equiv 1$) loại bỏ nguy cơ này. Đây chính là lý do đề bài yêu cầu safe prime.

4.5. Các ứng dụng khác

- **SSH:** trao đổi khóa phiên (`diffie-hellman-group14-sha256` , `curve25519-sha256`).
- **IPsec / IKEv2:** thỏa thuận khóa cho VPN, dùng các nhóm MODP của RFC 3526 (1536–8192 bit).
- **Signal / WhatsApp:** X3DH và Double Ratchet dựa trên DH trên đường cong (X25519), đổi khóa liên tục cho mỗi tin nhắn.

4.6. Tài liệu tham khảo

1. W. Diffie, M. Hellman, "New Directions in Cryptography", *IEEE Trans. Information Theory*, 22(6), 1976.
2. E. Rescorla, *RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3*, 2018.
3. D. Gillmor, *RFC 7919: Negotiated Finite Field Diffie-Hellman Ephemeral Parameters for TLS*, 2016.
4. D. Harkins, D. Carrel, *RFC 2409: The Internet Key Exchange (IKE)*, 1998 (Oakley Group 2, 1024 bit).
5. D. Adrian et al., "Imperfect Forward Secrecy: How Diffie-Hellman Fails in Practice", *ACM CCS 2015* (weakdh.org).
6. Giáo trình: S. Galbraith, *Mathematics of Public Key Cryptography*, chương 20 (liên kết trong đề bài).

MỤC 5: MÔ TẢ CHƯƠNG TRÌNH, KIỂM THỬ & HƯỚNG DẪN SỬ DỤNG

5.1. Môi trường phát triển và Yêu cầu hệ thống

Chương trình được thiết kế tương thích đa nền tảng (Cross-platform), hoạt động hoàn toàn chính xác trên cả Windows, Linux và macOS:

- **Hệ điều hành thử nghiệm:**
- Windows 10/11 (sử dụng môi trường MSYS2 MinGW-w64 UCRT64).
- Linux (Ubuntu 20.04 / 22.04 LTS hoặc Windows Subsystem for Linux - WSL).
- **Trình biên dịch:** `g++` (phiên bản GCC 7.0 trở lên, khuyến nghị GCC 11+ hoặc GCC 14/15).
- Yêu cầu hỗ trợ kiểu phần cứng mở rộng `__uint128_t` / `unsigned __int128`.
- **Chuẩn ngôn ngữ: C++14 trở xuống** (`-std=c++14`), tuân thủ nghiêm ngặt quy định không sử dụng tính năng của C++17/C++20 và chỉ dùng thư viện chuẩn C++ Standard Library.

5.2. Kiến trúc tổ chức mã nguồn (Unity Build)

Theo quy định của đề bài:

> *"Cấu trúc bài nộp: `project_01_source/*.cpp`"*

Để đảm bảo thư mục nộp bài chỉ chứa duy nhất các file mã nguồn `.cpp` mà không cần file header rời `.h` hay `Makefile`, nhóm áp dụng mô hình kiến trúc **Unity Build**:

```
project_01_source/
├── fips202.cpp     # Cài đặt lõi Keccak, hàm băm SHA-3 & hàm XOF SHAKE128 (FIPS 202)
├── prng.cpp        # Bộ sinh số ngẫu nhiên CSPRNG dùng SHAKE128, port rand_s cho Windows
├── bigint.cpp      # Kiểu dữ liệu số lớn big_t, các toán tử số học & chia Knuth D
├── dh.cpp          # Lũy thừa modulo, chọn phần tử sinh g và kiểm tra khóa công khai
├── prime.cpp       # Rejection sampling, Miller-Rabin, sàng đồng thời & sinh Safe Prime
└── main.cpp        # Điểm vào chương trình, xử lý tham số CLI & điều phối luồng Diffie-Hellman
```

- Mỗi module được viết độc lập trong một file `.cpp` có include guard (`#ifndef ...`).
- File `main.cpp` đóng vai trò là file tổng hợp, `#include` các file `.cpp` khác theo đúng thứ tự phụ thuộc.
- **Ưu điểm vượt trội**:
- Người chấm bài chỉ cần biên dịch đúng 1 file `main.cpp` duy nhất bằng 1 lệnh `g++`.
- Tận dụng tối đa tính năng tối ưu hóa toàn cục của trình biên dịch (Whole Program Optimization / Inlining) mà không cần bật LTO phức tạp.
- Loại bỏ hoàn toàn lỗi liên kết trùng lặp định nghĩa (ODR violation).

5.3. Hướng dẫn biên dịch chương trình

Mở terminal tại thư mục `project_01_source/` (hoặc thư mục gốc sau khi giải nén bài nộp):

```
cd project_01_source
g++ -std=c++14 -O2 -Wall -Wextra -Wpedantic -o dh main.cpp
```

Giải thích các cờ biên dịch:

- `-std=c++14`: Bắt buộc tuân thủ chuẩn C++14 theo đúng yêu cầu đề bài.

- `-O2`: Bật tối ưu hóa mức độ cao của trình biên dịch (tăng tốc độ thực thi các phép nhân số lớn và vòng lặp Miller-Rabin).
- `-Wall -Wextra -Wpedantic`: Bật toàn bộ cảnh báo kiểm tra lỗi cú pháp và kiểu dữ liệu (mã nguồn được đảm bảo biên dịch sạch tuyệt đối, không có warning).
- `-o dh`: Đặt tên file thực thi sinh ra là `dh` (hoặc `dh.exe` trên Windows).

5.4. Hướng dẫn chạy chương trình & Cấu hình tham số dòng lệnh

Chương trình cung cấp giao diện dòng lệnh linh hoạt với các tùy chọn tham số:

1. Xem hướng dẫn sử dụng

```
./dh -h
# hoặc:
./dh --help
```

2. Chạy mặc định (Cấu hình chuẩn của đề bài)

```
./dh
```

- Độ dài số nguyên tố p: **512 bit** (mặc định).
- Số vòng kiểm tra Miller-Rabin: **40 vòng**.
- Nguồn ngẫu nhiên: Lấy entropy trực tiếp từ hệ điều hành (`rand_s` trên Windows, `getrandom` trên Linux).

3. Tùy chỉnh độ lớn của số nguyên tố p (`--bits`)

Để kiểm nghiệm hiệu năng hoặc đáp ứng các tiêu chuẩn bảo mật cao hơn (ví dụ 768 bit, 1024 bit):

```
./dh --bits 768
# hoặc:
./dh --bits 1024
```

4. Tùy chỉnh số vòng kiểm tra Miller-Rabin (`--rounds`)

Xác suất một hợp số vượt qua kiểm tra Miller-Rabin sau k vòng không vượt quá 4^{-k}. Với $k = 40$, xác suất sai số là $4^{-40} \approx 8{,}27 \times 10^{-25}$ (đủ an toàn tuyệt đối trong thực tế). Để thay đổi số vòng kiểm tra:

```
./dh --bits 512 --rounds 50
```

5. Chạy với Seed cố định (`--seed`) để gỡ lỗi và tái lập kết quả

Đề bài cho phép dùng seed cố định phục vụ mục đích kiểm thử và tái hiện kết quả. Người dùng có thể truyền vào chuỗi hex 64 ký tự (tương ứng đúng 32 byte):

```
./dh --bits 512 --seed 0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef
```

5.5. Hướng dẫn thay đổi các tham số trong mã nguồn

Nếu muốn thay đổi cố định các giá trị mặc định trực tiếp trong mã nguồn:

1. **Thay đổi giới hạn kích thước bit tối đa (`BIG_LIMBS`)**: - Vị trí: Trong file `project_01_source/bigint.cpp` (dòng 23): `cpp static const size_t BIG_LIMBS = 32; // 32 limbs x 64 bit = 2048 bit` - Cấu trúc `big_t` hỗ trợ số nguyên lớn tới 2048 bit, đủ dung lượng chứa kết quả tích trung gian của các số 1024-bit. Nếu cần mở rộng lên 4096 bit, chỉ cần điều chỉnh `BIG_LIMBS = 64`.

2. **Thay đổi phần tử sinh g**: - Vị trí: Trong hàm `choose_generator()` tại `project_01_source/dh.cpp`: - Mặc định chọn $g = 4 = 2^2 \bmod p$, sinh nhóm con thặng dư bậc hai cấp $q = (p-1)/2$ để triệt tiêu hoàn toàn rủi ro rò rỉ parity của khóa bí mật.

5.6. Minh họa kết quả chạy thực tế (Terminal Output)

Kết quả thực thi thực tế của chương trình trên máy thử nghiệm với cấu hình chuẩn 512-bit:

```
============================================================
  DO AN 1: SO NGUYEN TO & TRAO DOI KHOA DIFFIE-HELLMAN
  Nhom 18 - Mon: Nhap mon ma hoa mat ma
============================================================
Cau hinh:
  - Do dai bit p: 512 bit
  - So vong Miller-Rabin: 40
  - PRNG Seed: Ngau nhien tu he dieu hanh (randombytes)
============================================================

[1] Dang sinh so nguyen to an toan p (512 bit)...
    => Hoan tat trong: 0.383135 giay
     p = 86aee1daa34b41b726b9eea8f1e664c3e1d4b249e10573d1c85a25bcb3197031f87779e9690c53c30ba9fdd1b7f20f71a5b6
ceba60f4052f1bff879cb072612f
    (p co dung 512 bit)

[2] Chon phan tu sinh g...
    g = 4
    Kiem tra g != 1 va g^q mod p == 1: DUNG (g sinh nhom con bac nguyen to q = (p-1)/2)

[3] Sinh khoa rieng cho Alice va Bob trong khoang [2, p - 2]...
     Khoa rieng Alice a = 4df54fa0252e372f66c186113e72ed087b3bbde7aa76c555750c9ccb150e692dc89f86095c60e6d0adf
d3b939ac74a2e89d0b1fee33b3a3dd88855cb1f0ce698
    Khoa rieng Bob   b = 687ea30dbbd2fde264573eba30078b4b1819dc13b76ad76b9421258660302bc4e4695fa2f7bb791beaa
dc88a655d87e98d718f5b41ee5196a0d906085ebf6bee

[4] Tinh gia tri cong khai:
    Alice gui A = g^a mod p = 3521abedb37bd83bb9983e48fbd0cef6ec72dfebabcbf1169ca4914572fc98e2211b885dce1bc4
db4888705cdd95054bfc30508ac5310aa74b2fb7d42326b9e0
    Bob   gui B = g^b mod p = 35a2bf8d0085bc85d4b067adeb1a2fe92d1cdda9667b3f39a346b77325b9271218f102cd6085c6
0db9378b6b3e2f42ddbba48ec328b7c390b5eed7db2a0fe0de
    Thoi gian tinh A, B: 0.001157 giay
    Kiem tra gia tri cong khai hop le (trong [2, p-2] va A^q = B^q = 1): DUNG

[5] Tinh bi mat chung:
    Bi mat chung Alice tinh duoc (s_A) = 5fd013cfbc23fa6fbbf0f3421a2baf01140d54ac9061ae0ef529f9829fed8646a29
85342b26f45290f10375b4d84a314c91a8a4aa8174f786708c575e476db0a
    Bi mat chung Bob   tinh duoc (s_B) = 5fd013cfbc23fa6fbbf0f3421a2baf01140d54ac9061ae0ef529f9829fed8646a29
85342b26f45290f10375b4d84a314c91a8a4aa8174f786708c575e476db0a
    Thoi gian tinh toan DH: 0.0014605 giay

============================================================
  [XAC MINH THANH CONG] Bi mat chung hai ben hoan toan TRUNG KHOP!
============================================================
```

MỤC 6: TỰ ĐÁNH GIÁ ƯU/NHƯỢC ĐIỂM, HƯỚNG CẢI THIỆN & PHÂN CÔNG CÔNG VIỆC

Đóng góp nội dung: Lâm Duy Minh (24120091), Vương Hữu Khang (24120070), Tạ Quang Mạnh (24120089)

6.1. Đánh giá phần Bộ sinh số ngẫu nhiên (PRNG), Kiểm tra nguyên tố & Sinh Safe Prime

Người thực hiện: Lâm Duy Minh (MSSV: 24120091)

1. Ưu điểm đã đạt được:

- **An toàn mật mã và tương thích đa nền tảng (Cross-platform CSPRNG):**
- Port thành công hàm lấy entropy hệ điều hành `randombytes()` sang Windows thông qua `rand_s()` (`RtlGenRandom`) song song với `getrandom()` trên Linux/macOS mà không cần phụ thuộc bất kỳ thư viện liên kết ngoài nào.
- Khắc phục triệt để các lỗi tiềm ẩn trong mã nguồn tham khảo ban đầu: mở rộng tham số `outlen` từ `uint8_t` lên `size_t`, ép kiểu tường minh trước khi dịch bit trong `prng_next_u16` / `prng_next_u32` để loại bỏ Undefined Behavior (tràn bit dấu `int` trong C++14), và nạp đủ 256 bit (32 byte) entropy cho trạng thái Sponge của SHAKE128.
- **Phân phối ngẫu nhiên đều tuyệt đối (Zero Modulo Bias):**
- Cài đặt thuật toán **Rejection Sampling** trong `random_range(ctx, lo, hi)` dựa trên mặt nạ bit tối tiểu $2^k - 1$, đảm bảo khóa riêng $a, b \in [2, p-2]$ và các cơ sở kiểm tra Miller-Rabin có phân phối đều 100%, không dùng phép chia lấy dư `%` gây chệch thống kê.
- **Tối ưu hóa tốc độ sinh Safe Prime bằng kiến trúc lọc 4 tầng:**
- Áp dụng **sàng đồng thời (Simultaneous Sieve)** trên cả q và $p = 2q + 1$ chỉ bằng 1 phép chia nhỏ `mod_small(q, r)` cho mỗi số nguyên tố lẻ $r < 2000$ (kiểm tra $q \bmod r \in \{0, (r-1)/2\}$), loại bỏ ngay $\sim 98{,}56\%$ ứng viên hợp số với chi phí tính toán không đáng kể.
- Đặt bước kiểm tra nhanh 1 vòng Miller-Rabin cơ sở 2 (`miller_rabin_base2`) cho cả q và p **trước** khi chạy đủ 40 vòng Miller-Rabin cơ sở ngẫu nhiên, giúp giảm số lần chạy Miller-Rabin 40 vòng từ hàng chục lần xuống đúng **1 lần duy nhất** cho mỗi safe prime tìm được.
- Thời gian sinh safe prime 512 bit chỉ mất trung bình **~0,26 – 0,38 giây** và 1024 bit chỉ mất **~4,5 giây**, vượt xa kỳ vọng ban đầu của nhóm.

2. Hạn chế tồn đọng:

- **Chưa chống tấn công kênh kề theo thời gian (Timing Side-Channel):**
- Các vòng lặp trong `modular_exponentiation` và `is_probable_prime` hiện sử dụng nhánh điều kiện phụ thuộc vào bit của số mũ/khóa bí mật và thoát sớm (early exit) khi phát hiện hợp số. Trong môi trường thực tế nếu kẻ tấn công đo được chính xác thời gian thực thi hoặc năng lượng tiêu thụ, một phần thông tin về số mũ có thể bị rò rỉ.
- **Sinh ứng viên tuần tự trên một luồng (Single-threaded):**
- Quá trình thử các ứng viên q trong `generate_safe_prime` hiện chạy trên 1 luồng CPU duy nhất. Khi tăng kích thước lên 2048 bit hoặc 4096 bit, số lượng ứng viên cần thử tăng lên hàng trăm nghìn, làm thời gian chờ trên 1 nhân kéo dài.
- **Chưa tự động tái nạp entropy (Reseeding):**
- Bộ sinh SHAKE128 hiện được khởi tạo seed 1 lần khi bắt đầu chương trình. Mặc dù dung lượng $c = 256$ bit của Keccak cho phép xuất hàng tỷ byte an toàn, một CSPRNG chuẩn công nghiệp (như NIST SP 800-90A) thường kết hợp cơ chế tự động nạp lại entropy (reseed) định kỳ sau một ngưỡng dữ liệu nhất định để đảm bảo tính chống xâm phạm trạng thái (prediction resistance).

3. Hướng cải thiện trong tương lai:

- **Tìm kiếm bước nhảy tăng dần trên bảng dư (Incremental Sieve Table):**

- Thay vì sinh mới hoàn toàn ứng viên q từ PRNG sau mỗi lần bị loại bởi sàng, có thể sinh một số ngẫu nhiên gốc q_0, lưu bảng số dư `rem[i] = q_0 % SMALL_PRIMES[i]`, sau đó tăng $q \leftarrow q + 2$ và cập nhật bảng số dư bằng phép cộng đơn giản `rem[i] = (rem[i] + 2) % r` trong $O(1)$ mà không cần gọi lại `mod_small`.
- **Song song hóa đa luồng (`std::thread` trong C++11/14):**
- Khởi tạo T luồng phần cứng (`std::thread::hardware_concurrency()`), mỗi luồng giữ một ngữ cảnh `prng_ctx` được phân tách miền (domain-separated seed) để tìm kiếm ứng viên safe prime độc lập và dừng ngay khi luồng đầu tiên tìm thấy, giúp tăng tốc gần tuyến tính theo số nhân CPU cho kích thước 2048–4096 bit.
- **Bổ sung kiểm tra Lucas (Baillie–PSW Primality Test):**
- Kết hợp 1 vòng Miller-Rabin cơ sở 2 với 1 phép kiểm tra giả nguyên tố Lucas mạnh (Strong Lucas Probable Prime Test) theo chuẩn FIPS 186-5; cho đến nay chưa tìm thấy bất kỳ hợp số nào vượt qua đồng thời cả hai phép kiểm tra này.

6.2. Đánh giá phần Hệ thống xử lý số lớn (Big Integer) & Thuật toán phép chia Knuth D

Người thực hiện: Vương Hữu Khang (MSSV: 24120070) & Tạ Quang Mạnh (MSSV: 24120089)

1. Ưu điểm đã đạt được:

- **Cấu trúc dữ liệu tĩnh (Zero Heap Allocation):**
- Kiểu `big_t` được cấp phát hoàn toàn trên Stack với kích thước cố định 2048 bit (32 từ nhớ 64-bit `uint64_t`). Tuyệt đối không dùng cấp phát động (`malloc`, `new`, `std::vector`), không gây phân mảnh bộ nhớ (heap fragmentation), tận dụng tối đa bộ đệm L1/L2 Cache của CPU.
- **Thuật toán nhân trường học tối ưu carry bằng kiểu 128-bit:**
- Cài đặt phép nhân $O(n^2)$ sử dụng kiểu nội tại `__int128_t` / `unsigned __int128` có sẵn trên GCC/Clang 64-bit, tích lũy nhớ (carry) trực tiếp trong 1 thao tác duy nhất mà không cần tách từ 32-bit, đạt hiệu năng tối đa trên vi kiến trúc x86_64.
- **Thuật toán chia Knuth Algorithm D (The Art of Computer Programming, Vol 2):**
- Thay thế hoàn toàn phép chia nhị phân (dịch bit - so sánh - trừ) tốn kém bằng thuật toán chia nhiều từ Knuth D:
  - Bước chuẩn hóa (Normalization): Dịch trái số chia V và số bị chia U sao cho bit cao nhất của từ nhớ đầu tiên $v_{n-1} \geq 2^{63}$, giúp độ chính xác của ước lượng thương $\hat{q}$ đạt tới 100% hoặc chỉ lệch tối đa 1 đơn vị.
  - Ước lượng thương và hiệu chỉnh: Tính thương thử $\hat{q} = \lfloor (u_j B + u_{j+1}) / v_1 \rfloor$ và hiệu chỉnh thông qua điều kiện $v_2 \hat{q} > (u_j B + u_{j+1} - \hat{q} v_1) B + u_{j+2}$.
  - Tăng tốc phép nhân - trừ và phục hồi nếu có mượn âm (Add-back step).
- Tích hợp đường tắt chia ngắn (Short Division) cho trường hợp số chia chỉ có 1 limb 64-bit và phép chia cho lũy thừa của 2 bằng dịch bit phải `>>`.
- Giảm thời gian thực hiện phép chia lấy dư 512-bit từ $116\ \mu s$ **xuống còn** $0{,}91\ \mu s$ **(tăng tốc hơn 100 lần)**.
- **Độ tin cậy được bảo chứng:**
- Đã chạy kiểm thử áp lực đối chiếu kết quả với thư viện số nguyên lớn của Python (`tests/crosscheck_div_stress.py`) với hơn **50.000 ca kiểm thử ngẫu nhiên và ca biên ngặt nghèo**, đạt tỷ lệ chính xác 100%, không có bất kỳ sai lệch nào.
- Cài đặt đầy đủ các hàm hỗ trợ theo pseudocode của giảng viên: `AddMod`, `MultMod`, `ExpMod`.

2. Hạn chế tồn đọng:

- **Giới hạn kích thước tối đa 2048 bit:**
- Cấu trúc tĩnh cố định `BIG_LIMBS = 32` (2048 bit). Mặc dù đáp ứng hoàn hảo cho đề tài (yêu cầu ≥ 512 bit, an toàn thực tế 1024-2048 bit), thư viện chưa hỗ trợ số lớn động không giới hạn (Arbitrary-precision BigInt).
- **Chưa tối ưu phép nhân Karatsuba / Toom-Cook / FFT:**

- Phép nhân hiện tại có độ phức tạp trường học $O(n^2)$. Với kích thước 512-1024 bit (8 đến 16 từ nhớ 64-bit), thuật toán trường học thực tế vẫn chạy rất nhanh nhờ ít overhead; tuy nhiên nếu mở rộng lên 4096 hay 8192 bit, Karatsuba ($O(n^{1,58})$) sẽ mang lại lợi thế vượt trội.
- **Chưa triển khai biểu diễn Montgomery (Montgomery Representation):**
- Trong phép tính lũy thừa modulo $a^e \bmod m$, mỗi phép nhân hiện tại vẫn phải gọi một phép chia lấy dư `%` bằng thuật toán Knuth D ($O(n^2)$). Cài đặt phép nhân Montgomery sẽ cho phép chuyển đổi sang miền Montgomery và thực hiện rút gọn modulo (Montgomery Reduction) chỉ bằng các phép nhân và dịch bit, không cần phép chia nào.

3. Hướng cải thiện trong tương lai:

- **Tích hợp phép nhân Montgomery:**
- Chuyển đổi $x \mapsto \tilde{x} = x \cdot R \bmod m$ với $R = 2^{64 \cdot n}$, áp dụng thuật toán CIOS (Coarsely Integrated Operand Scanning) để tối ưu hóa triệt để hàm `modular_exponentiation`.
- **Tận dụng chỉ thị phần cứng SIMD / Crypto Extensions:**
- Sử dụng các lệnh mở rộng x86_64 hiện đại như `MULX` (nhân không ảnh hưởng cờ), `ADCX` / `ADOX` (cộng đồng thời hai chuỗi nhớ trên hai cờ CF và OF riêng biệt của Intel Broadwell trở lên) để nhân đôi băng thông tính toán số học lớn.

6.3. Đánh giá phần Giao thức Trao đổi khóa Diffie-Hellman & Lũy thừa Modulo

Người thực hiện: Tạ Quang Mạnh (MSSV: 24120089) & Vương Hữu Khang (MSSV: 24120070)

1. Ưu điểm đã đạt được:

- **Tối ưu hóa Lũy thừa modulo bằng Square-and-Multiply:**
- Triển khai thuật toán bình phương và nhân liên tiếp (Left-to-Right Binary Exponentiation), giảm số lượng phép nhân modulo từ $O(e)$ xuống còn $O(\log e)$ (tối đa $2 \log_2 e$ phép nhân, trung bình $1{,}5 \log_2 e$).
- Đối chiếu độ chính xác với hàm chuẩn `pow(base, exp, mod)` của Python qua 4.452 trường hợp thử nghiệm: kết quả khớp tuyệt đối 100%.
- **Lựa chọn phần tử sinh an toàn trên nhóm con cấp nguyên tố:**
- Không chọn $g = 2$ tùy tiện khi chưa chứng minh bậc; thay vào đó, chương trình chọn $g = 4 = 2^2 \bmod p$, là phần tử sinh chắc chắn của nhóm con thặng dư bậc hai Q_p có cấp nguyên tố $q = (p-1)/2$.
- Mọi phần tử trong nhóm con đều có cấp lớn q, triệt tiêu hoàn toàn khả năng rò rỉ parity của khóa riêng a và làm vô hiệu hóa tấn công Pohlig-Hellman.
- **Xác thực khóa công khai chặt chẽ (Public Key Validation):**
- Hàm `is_valid_public_value(v, p)` kiểm tra đầy đủ cả 2 điều kiện:
  1. $2 \le v \le p - 2$.
  2. $v^q \equiv 1 \pmod p$ (xác nhận v nằm đúng trong nhóm con cấp q, không bị ép vào nhóm con nhỏ cấp 1 hoặc 2 do kẻ tấn công Man-in-the-Middle gửi đến).

2. Hạn chế tồn đọng:

- **Nhạy cảm với tấn công kênh kề thời gian (Timing Analysis / SPA):**
- Trong vòng lặp Square-and-Multiply, bước nhân `res = (res * base) % mod` chỉ được thực thi khi bit của số mũ bằng 1. Sự chênh lệch thời gian giữa các bit 0 (chỉ bình phương) và bit 1 (bình phương + nhân) có thể bị kẻ tấn công đo đạc bằng dao động ký hoặc bộ đếm chu kỳ để khôi phục từng bit khóa riêng.
- **Chưa có cơ chế xác thực danh tính hai đầu (Unauthenticated DH):**
- Giao thức Diffie-Hellman cổ điển nguyên bản không có chữ ký số xác thực danh tính của Alice và Bob, do đó dễ bị tấn công xen giữa (Man-in-the-Middle) nếu kênh truyền bị kiểm soát hoàn toàn bởi kẻ tấn công. Trong thực tế, DH luôn phải kết hợp với chứng chỉ số X.509 và chữ ký số RSA/ECDSA (như trong TLS 1.3).

3. Hướng cải thiện trong tương lai:

- **Triển khai thuật toán Montgomery Ladder (Constant-time Exponentiation):**
- Cài đặt Montgomery Ladder đảm bảo với mọi bit của số mũ (dù là 0 hay 1), chương trình đều thực hiện đúng 1 phép bình phương và 1 phép nhân. Kết hợp kỹ thuật thanh ghi tạm (dummy variable) để loại bỏ hoàn toàn dấu vết thời gian và năng lượng tiêu thụ (chống SPA/DPA).
- **Mở rộng sang Elliptic Curve Diffie-Hellman (ECDH):**
- Triển khai trao đổi khóa trên đường cong Elliptic (ví dụ Curve25519 theo chuẩn RFC 7748). ECDH mang lại độ an toàn tương đương (128-bit security) nhưng chỉ cần khóa 256-bit (thay vì 2048-bit hoặc 3072-bit trong DH cổ điển), giúp tốc độ tính toán nhanh hơn hàng chục lần và giảm kích thước gói tin trao đổi qua mạng.

6.4. Bảng tự đánh giá mức độ hoàn thành & Đóng góp của từng thành viên

Cả 3 thành viên trong nhóm đều thể hiện tinh thần trách nhiệm cao, chủ động nghiên cứu cơ sở lý thuyết, đóng góp mã nguồn chất lượng, kiểm thử chéo nghiêm ngặt và phối hợp viết báo cáo đầy đủ, hoàn thành trước thời hạn quy định.

STT	Họ và tên	MSSV	Nhiệm vụ chính được phân công	Mức độ hoàn thành	Tỷ lệ đóng góp
1	**Vương Hữu Khang**	24120070	Kiến trúc tổng thể; Cài đặt BigInt cơ sở (`+` , `-` , `*` , chuyển đổi hex/dec); Cài đặt PRNG Keccak/SHAKE128; Viết Test suite, Pseudocode modular arithmetic (`AddMod` , `MultMod` , `ExpMod`); Soạn thảo Mục 1 (chính), Mục 5, tổng hợp Mục 6 và xuất báo cáo PDF.	**100%**	**33.3%**
2	**Tạ Quang Mạnh**	24120089	Cài đặt phép chia số lớn Knuth Algorithm D (`divmod` , `/` , `%`); Cài đặt Diffie-Hellman (`ExpMod` , `choose_generator` , `is_valid_public_value`); Viết Test suite phép chia, stress test đối chiếu Python; Soạn thảo Mục 1.5, 1.6 và Mục 4 (Ứng dụng thực tế của DH trong TLS 1.3).	**100%**	**33.3%**
3	**Lâm Duy Minh**	24120091	Port PRNG entropy sang Windows (`rand_s`); Cài đặt Rejection Sampling (`random_range`); Cài đặt Miller-Rabin (`uint64_t` và `big_t`); Cài đặt Sàng đồng thời 302 số nguyên tố nhỏ & Sinh Safe Prime 4 tầng; Viết Benchmark thực nghiệm; Soạn thảo Mục 2, Mục 3 (điểm cộng) và Mục 6.1.	**100%**	**33.3%**